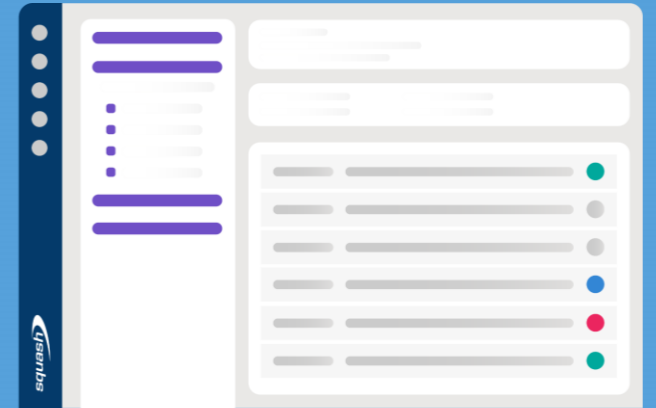


Les fondamentaux du test fonctionnel



Je m'appelle ...

Mon expérience ...

Je recherche dans cette formation ...

I. Introduction

1. La Qualité
2. Le test logiciel
3. Le cycle de vie d'un projet logiciel
4. Processus de test

→ Pourquoi on teste ?

II. Présentation générale de Squash TM

1. La suite Squash
2. Squash TM

→ Quels outils pour tester ?

III. Le patrimoine d'exigences

1. La notion d'exigence
2. Identifier une exigence
3. Formaliser une exigence
4. Qualifier une exigence

→ Qu'est-ce qu'on teste ?

IV. Le référentiel des cas de test

1. La notion de cas de test
2. La notion de jeu de données
3. Technique de conception des tests
4. Identifier les tests de non-régression
5. La couverture fonctionnelle

→ Avec quoi on le teste ?

→ Comment on le teste ?

V. L'exécution des tests

1. Concevoir un plan d'exécution optimisé
2. Exécuter les tests
3. Comprendre la notion d'anomalie

→ Comment on organise les tests ?

VI. Pilotage et bilan

1. Définitions
2. Les tableaux de bord
3. Les graphiques
4. Les bilans

→ Comment on gère les résultats des Tests ?

Introduction

1. La Qualité

- a) Définitions
- b) Histoire
- c) Qualité/coût/délais

2. Le test logiciel

- a) Définitions
- b) Histoire
- c) Indépendance & biais
- d) Les 7 principes du Test

3. Le cycle de vie d'un projet logiciel

- a) L'approche « Cycle en V »
- b) L'approche « Agile »
- c) Les types de tests

4. Processus de test

- a) Définitions
- b) Environnement
- c) Équipe

1. La Qualité

- La notion de Qualité est au cœur du projet logiciel :



Les activités de test (la **Recette**)
consistent à **Évaluer**
la **Qualité observée**
par rapport à la **Qualité attendue**.

- Qu'est-ce-que, selon **vous**, la Qualité ?
- Nous verrons ensuite comment définir un logiciel « de bonne qualité ».



Définition objective,
indépendante du
contexte

- **La Qualité possède une définition objective, générale (celle du dictionnaire) :**

- Aspect, manière d'être de quelque chose, ensemble des modalités sous lesquelles quelque chose se présente : *Photographe attentif à la qualité de la lumière.*
- Ensemble des caractères, des propriétés qui font que quelque chose correspond bien ou mal à sa nature, à ce qu'on en attend : *Du papier de qualité moyenne.*
- Ce qui rend quelque chose supérieur à la moyenne : Préférer la qualité à la quantité.
- Chacun des aspects positifs de quelque chose qui font qu'il correspond au mieux à ce qu'on en attend : *Cette voiture a de nombreuses qualités.*
- Trait de caractère, manière de faire, d'être que l'on juge positivement : *Qualités morales. Des qualités de cœur.*

- Mais il en existe aussi des définitions plus subjectives :

- Aspect, manière d'être de quelque chose, ensemble des modalités sous lesquelles quelque chose se présente : *Photographe attentif à la qualité de la lumière.*

Définitions **subjectives**, qui impliquent une définition de **l'objectif** et une **évaluation** du **résultat**.

Elles impliquent **de se mettre à la place de l'utilisateur final**.

- Ensemble des caractères, des propriétés qui font que quelque chose correspond bien ou mal à sa nature, à ce qu'on en attend : *Du papier de qualité moyenne.*
- Ce qui rend quelque chose supérieur à la moyenne : *Préférer la qualité à la quantité.*
- Chacun des aspects positifs de quelque chose qui font qu'il correspond au mieux à ce qu'on en attend : *Cette voiture a de nombreuses qualités.*
- Trait de caractère, manière de faire, d'être que l'on juge positivement : *Qualités morales. Des qualités de cœur.*



L'objectif de viser une **Qualité supérieure** apportant plus de « **Business Value** » (avantage commercial) n'est pas récent :

« Si nos fabriques parviennent à obtenir la qualité supérieure de nos produits, alors les étrangers trouveront avantage à se fournir en France et leur argent affluera dans tout le royaume. »

Colbert – 1664



- La Qualité devient un **élément constitutif de la production** avec le [Taylorisme](#) dans les années 1910.
- La mise au point de **processus industriels reproductibles et généralisables** permet d'obtenir une production de **qualité plus constante**.



- **1930-1940 Naissance du contrôle qualité** appliqué à la production industrielle
 - Indicateurs : Méthodes statistiques, échantillonnage.
- **1940-1950 Niveau de qualité acceptable**
 - Indicateurs : % d'éléments défectueux, Tolérance acceptée.
- **1950-1960 Notion de fiabilité**
 - Indicateurs : Conditions de service définies, durée déterminée.
- **1960-1980 Contrôle de la qualité totale** ([Total Quality Control](#))
 - Indicateurs : Prévention, action & réaction, responsabilité du management.
- **1980-1990 Orientation vers le client**
 - Indicateurs : Satisfaction, séduction, différenciation (produit).
 - Slogan : « La qualité est l'affaire de tous ! » ([W. Edwards Deming](#))
- **1990-ce jour Management total de la qualité** (Total Quality Management*)
 - Responsabilité de tous les acteurs, mais la direction du Projet est responsable à 100% des problèmes de qualité et de leur persistance.



L'âge du TRI :
Fabrication
Tri
Rejet des « mauvais »

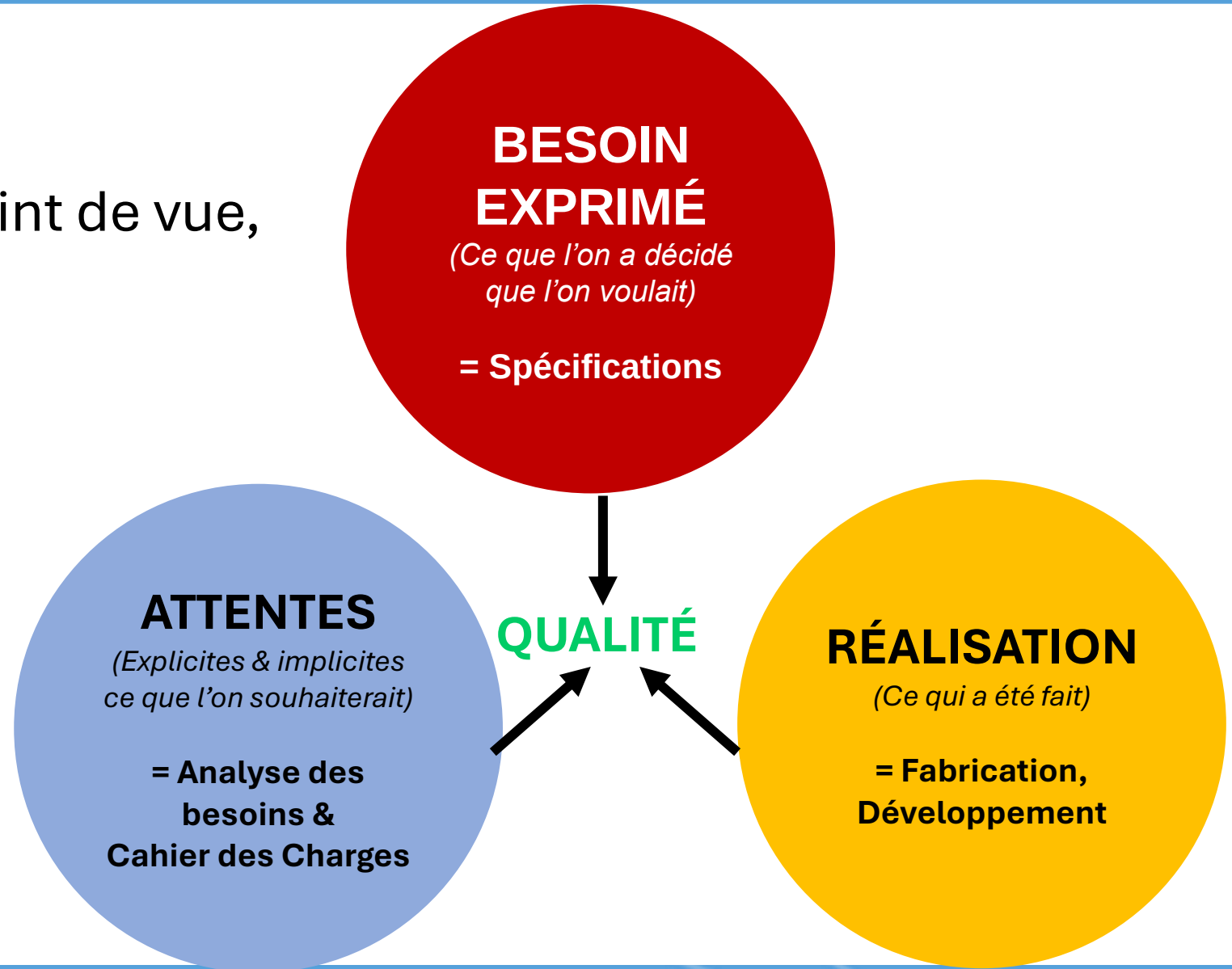
L'âge du CONTRÔLE :
Contrôle
au cours de la fabrication
Action corrective si déviation

L'âge de
L'AMÉLIORATION CONTINUE
Contrôle du processus
en amont
KPI / Feedback
PDCA

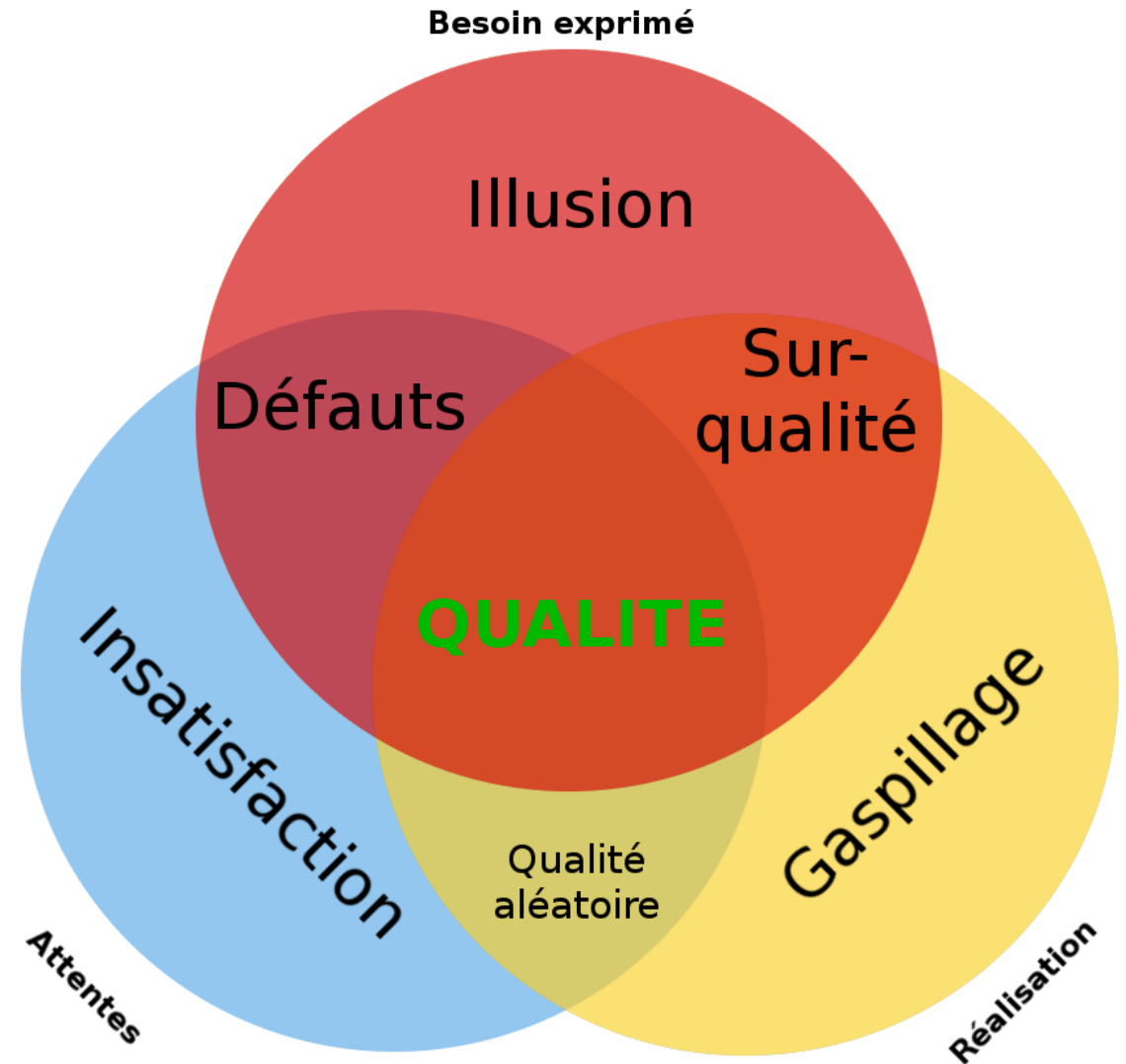


- Définitions ISTQB de la qualité et qualification :
 - **Qualité** : Degré par lequel un composant, système ou processus atteint des **exigences spécifiées** et/ou des **besoins** ou **attentes** des clients ou utilisateurs. [in ISO 24765]
 - **Qualification** : Le processus consistant à démontrer la capacité à satisfaire les **exigences spécifiées**. Note : le terme « qualifié » est utilisé pour désigner le statut correspondant. [in ISO 9000]

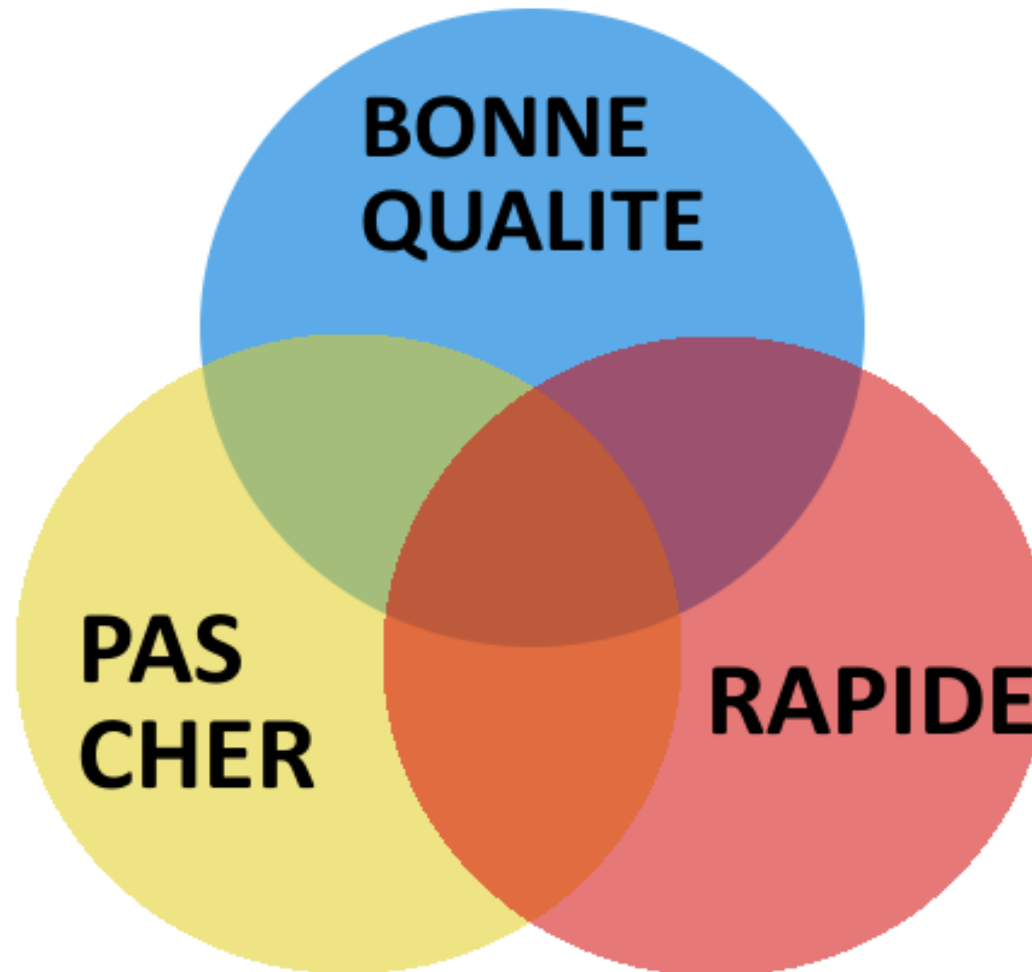
Si la Qualité dépend du point de vue,
la **Qualité perçue**
se situe au **croisement**
des **attentes**,
du **besoin exprimé**
et de leur **réalisation**.



- La **Qualité produite** se situe à l'intersection **réelle** de ces trois composantes.
- Les attentes *non exprimées dans les besoins* ne seront **pas développées ni réalisées** et génèreront de l'**insatisfaction** chez les utilisateurs.
- Des **attentes des utilisateurs, spécifiées mais non développées, ou « mal » développées**, constitueront des **défauts**.
- Etc.



- Le rêve du client :

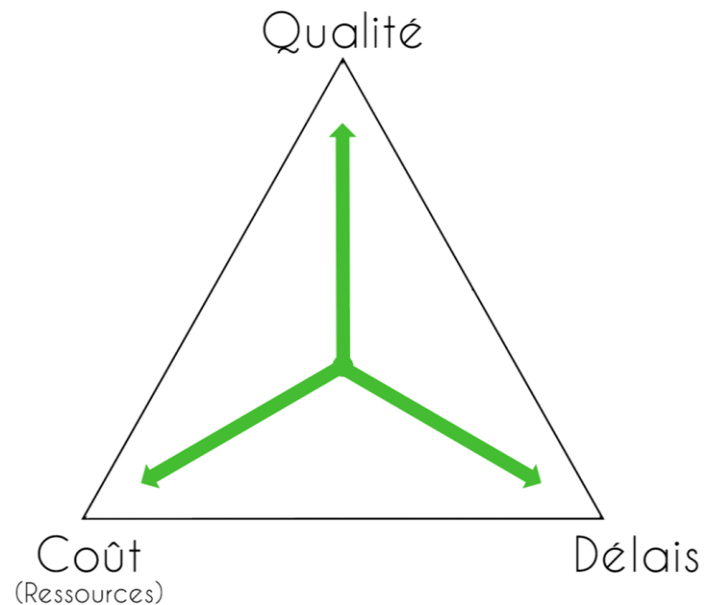


- La réponse (honnête) du fournisseur :



NB : "GRATUIT" n'est pas une option.

Qualité, coût et délai sont trois notions fondamentales et interdépendantes qu'il faut connaître et évaluer dans un projet. Pour représenter leur interdépendance, on parle du **triangle Qualité/Coût/Délai**.



- Si on souhaite **améliorer la qualité**, cela risque **d'augmenter le coût** ou de **rallonger le délai** (ou les deux).
 - Si vous devez **réduire les coûts**, il se peut alors que la **qualité en pâtisse** ou que le **délai doive être revu à la hausse** (moins de personnel, moins de ressources, etc.).
 - Si vous avez une **date butoir courte**, il vous faudra souvent **plus de ressources** (donc un coût plus élevé) ou accepter de **limiter certains critères de qualité**.
-
- Agir sur un paramètre conditionne les 2 autres.
 - Dans le monde réel, ce sont souvent les délais qui priment et qui impacteront le plus les activités de test, En effet, la recette arrive en fin de chaîne, avant la mise en production (MEP).



Le mythe du mois-homme

Le mois-homme représente le travail d'un homme pendant un mois.
(On utilise plus souvent le jour-homme, abrégé en j.H (***pas j/h !***))

Le seul fait d'utiliser cette unité tend à faire croire que le travail d'une seule personne pendant n mois peut parfaitement être réalisé par n personnes pendant un seul mois ;

En suivant cette idée, on pourrait diviser les temps de développement par deux en mettant à l'œuvre deux fois plus de personnel.

→ Est-ce réaliste ? Quelles limites peut-il y avoir ? L'œuf sur le plat sera-t-il plus vite prêt si on est 10 en cuisine ?

→ Le Mythe du mois-homme (titre original : *The Mythical Man-Month: Essays on Software Engineering*) est un livre de Frederick Brooks considéré comme un classique dans le domaine du génie logiciel.

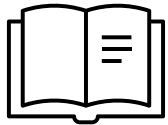
[Voir l'article sur Wikipedia](#)



WIKIPÉDIA
L'encyclopédie libre



Réflexion : Le mythe du mois-homme



« Neuf femmes ne font pas un enfant en un mois »

Citation de Frederick Brooks dans son livre *The Mythical Man-Month: Essays on Software Engineering*

Les projets de programmation complexes ne peuvent pas être parfaitement divisés en tâches discrètes qui peuvent être travaillées sans communication entre les travailleurs et sans établir un ensemble d'interrelations complexes entre les tâches et les travailleurs qui les exécutent.

L'ajout de nouvelles ressources entraîne :

- Un temps de montée en compétence
(comprendre le contexte, le sujet, les objectifs...)
- L'augmentation du temps de communication
- La diminution de la productivité par personne



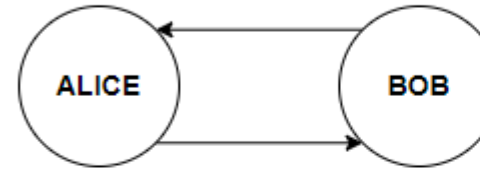


Communication : contraintes et limites

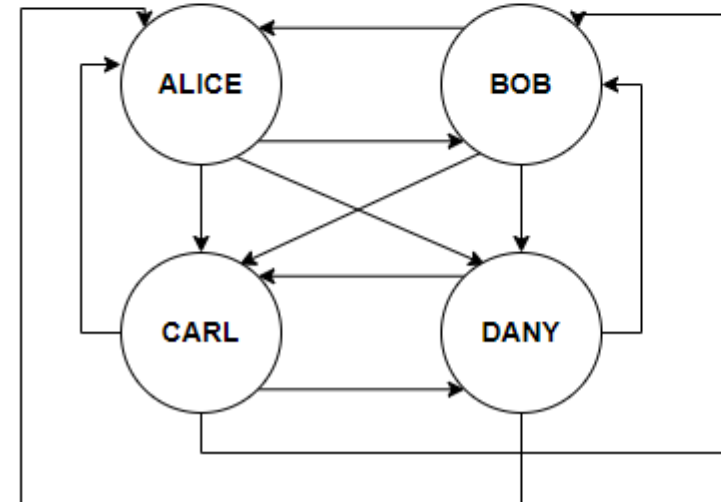
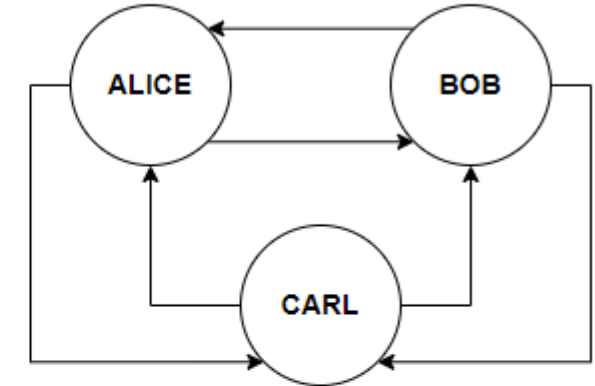
L'augmentation du temps et de la complexité de la communication est exponentiellement proportionnelle au nombre d'émetteurs/récepteurs.

De plus, **chaque canal de communication** court le risque d'être « brouillé » par des parasites (maladresses dans l'expression, mauvaise compréhension, manque d'écoute active, manque de feedback ...)

Avec 2
émetteurs/récepteurs :
2 voies

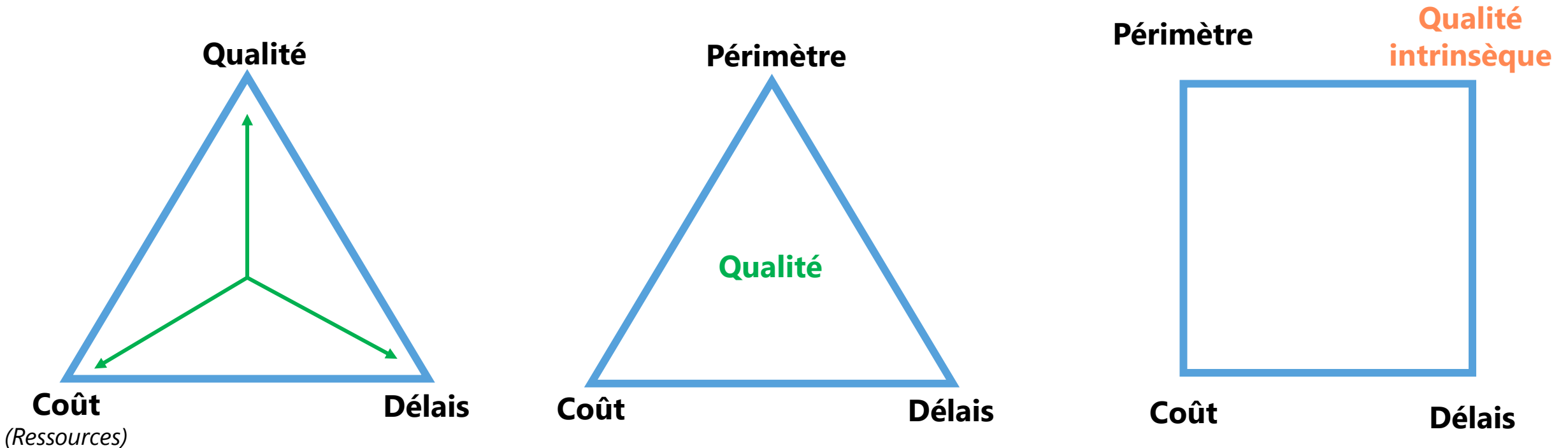


Avec 3
émetteurs/récepteurs :
6 voies

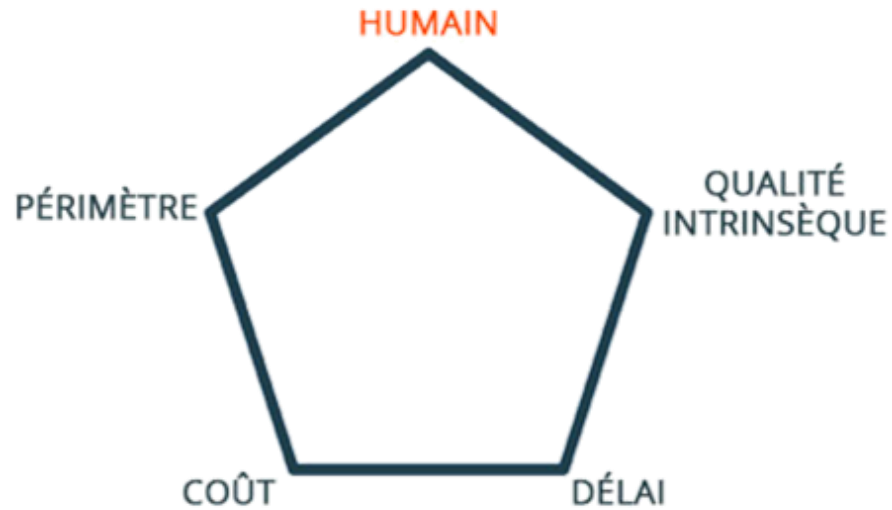


Avec 4
émetteurs/récepteurs :
12 voies

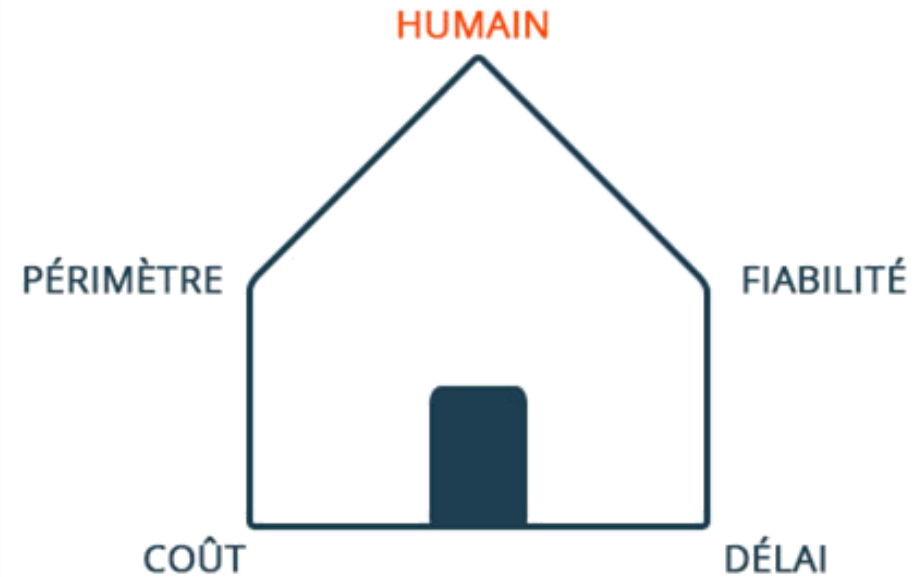
- Le triangle Qualité/Coût/Délai peut aussi prendre en compte un autre paramètre, notamment dans les projets Agile : le **Périmètre**. On peut alors représenter la qualité au centre du triangle, comme livrable cadré par les trois paramètres du triangle.
- Avec le **Carré de la Qualité**, on traitera séparément le **Périmètre** et la **Qualité intrinsèque** (càd, le niveau de qualité de ce qui aura été réalisé).



- Un autre paramètre essentiel : **l'Humain**.



***Le Pentagone de la Qualité,
dit aussi 'Diamant de la Qualité'***



... et sa métaphore

- En prenant ce paramètre en compte, (notamment en contexte **Agile**), on se donne la capacité de :
 - Maintenir un **rythme soutenable indéfiniment**, base du bien-être des équipes
 - Réduire la charge des systèmes et donc d'**accueillir favorablement les changements**
 - Se donner du **temps de recul pour améliorer** le système dans son ensemble

Le coût de la **Non-Qualité** :



- La non-qualité est l'écart global constaté entre la qualité visée et la qualité effectivement obtenue (NF X 50-120).

NO QUALITY

- **Plus tard est découvert le défaut, plus coûteuse est sa correction :**
 - en phase de développement : coût = 1
 - en phase d'intégration : coût = 10
 - en phase de recette : coût = 100
 - en phase d'exploitation : coût > 1000

Un exemple : Le 30 janvier 2024, Toyota rappelle **1 004 950** exemplaires de Corolla dans le monde dans le cadre d'une opération technique de sécurité.

Celle-ci vise à corriger un problème visant le **système de freinage**: la diminution ou la perte d'assistance du système entraîne un allongement de la distance d'arrêt et peut **engendrer un accident**.

Cette anomalie est due à un **mauvais calibrage de l'unité de commande électronique qui gère l'antipatinage**. Pour pallier le problème, une **reprogrammation de l'actionneur de frein** doit être opérée à l'atelier.

Imaginez le coût ... multiplié par 1 million ...

- La **sur-qualité** a elle aussi un coût !

La **sur-qualité** apparaît quand le **niveau de qualité optimal** génère un **coût inadéquat**.

« *Il faut savoir jusqu'où ne pas aller trop loin* ». (Jean Cocteau)

- Définition ISTQB :

- **Coût de la qualité** : Le coût total imputé aux activités et problèmes liés à la qualité, souvent divisé en
 - Coûts de **prévention**
 - Coûts d'**estimation**
 - Coûts des **défaillances internes** (*p. ex., pour la MOE, la non-qualité des TU/TI entraîne des coûts supplémentaires (charge en j-H) en recette*)
 - Coûts **des défaillances externes**. (**Coûts** résultant de **défauts** qui sont constatés **après la livraison du produit ou pendant la fourniture du service**. Ils comprennent habituellement les coûts de **garantie**, de **retour** du produit. Ils comprennent également les **coûts liés aux conséquences de la non-qualité** fournie.

- **Il faut donc se poser la question :**
« Qu'est-ce qu'une application de bonne qualité ? »
- Une application de bonne qualité :
 - Répond aux **Spécifications** qui ont guidé son développement
 - Fonctionne de la façon **attendue**
 - Peut être déployée en **conservant** ses caractéristiques
- La qualité logicielle est définie dans la norme ISO* 25010
(mise à jour de la norme ISO 9126) .
- [Voir sur le Moodle](#) « Les normes ISO sur la Qualité Logicielle »

* *ISO : International Organization for Standardization*



Quelques définitions fondamentales selon ISO 9126 :



- **Logiciel**

Programmes, procédures, règles et toute documentation associés relatifs au fonctionnement d'un ensemble de traitements informatiques.

- **Qualité logicielle**

Ensemble des traits et des caractéristiques d'un produit logiciel portant sur son aptitude à satisfaire des besoins exprimés ou implicites.

- **Caractéristiques de qualité**

Ensemble d'attributs d'un produit logiciel avec lequel on décrit et on évalue sa qualité. Une caractéristique de qualité d'un logiciel peut être subdivisée en de multiples niveaux de sous-caractéristiques.

- **Métrique**

Échelle quantitative et méthode que l'on peut utiliser pour déterminer la valeur d'une propriété d'un produit logiciel que l'on peut relier aux caractéristiques de qualité.

2. Le test logiciel



« Je suis très à cheval sur les principes mais très mauvais cavalier ».

Grégoire Lacroix

« La faute est dans les moyens bien plus dans les principes. »

Napoléon Bonaparte

- **Les textes normatifs définissant la Qualité Logicielle ont pour objectif de :**

- **Uniformiser et harmoniser les pratiques :**

Elles permettent aux entreprises, organisations de développement et de test, ainsi qu'à ceux qui la pratiquent de parler le même langage technique et organisationnel, facilitant ainsi les échanges et les collaborations à l'échelle internationale.

- **Garantir la qualité et la fiabilité :**

En s'appuyant sur ces référentiels, les parties prenantes peuvent assurer que leurs produits, services ou processus répondent à des critères reconnus en matière de qualité, de sécurité ou de performance.

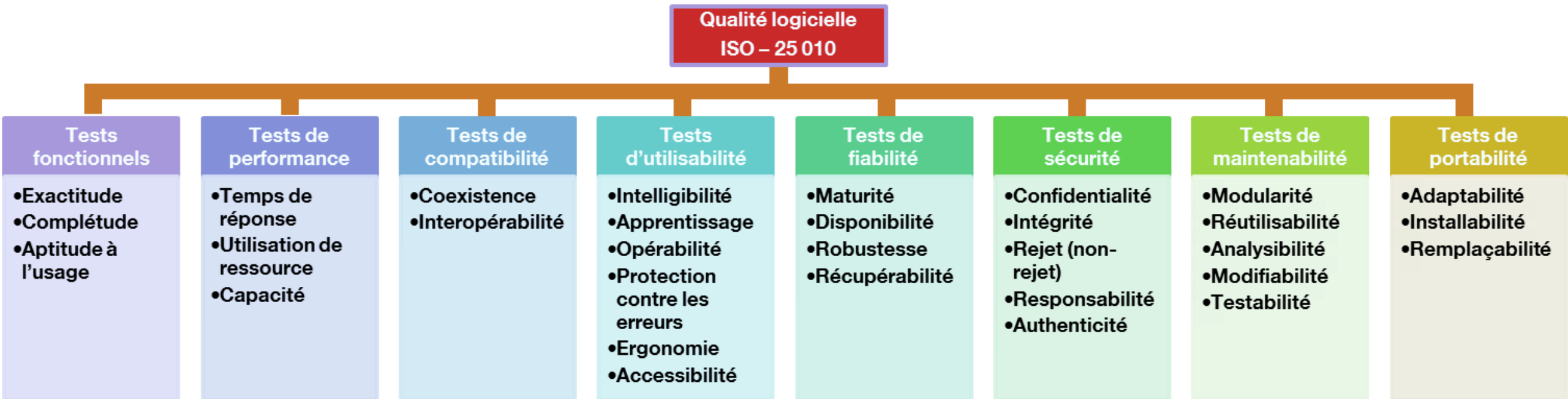
- **Renforcer la confiance :**

L'application des normes et des référentiels (comme ISTQB) signale aux clients, partenaires et autorités que l'organisation applique des pratiques validées, ce qui accroît la crédibilité et la confiance sur le produit et son évaluation.

- **Encourager l'amélioration continue :**

Ces normes et référentiels, en constante évolution, incitent les structures à évaluer régulièrement leurs processus et à les optimiser de façon continue.

La norme ISO 25 010 (mise à jour de la norme ISO 9126) définit un modèle pour l'évaluation de la qualité logicielle, en fonction de **8 familles de tests**.*



* Cette taxonomie des tests est spécifique à ISO 25 010.
ISTQB organise les familles de test un peu différemment.

- **La Recette :**

La recette d'un logiciel est l'activité qui consiste à vérifier que la qualité d'une application est bien au niveau de :

- ce qui est souhaité dans le cahier des charges,
- et ce qui spécifié dans les spécifications.

- **L'objectif :** la validation de la qualité demandée du logiciel.

- La recette recouvre la **vérification de conformité** et la **recherche et la découverte d'anomalies**.

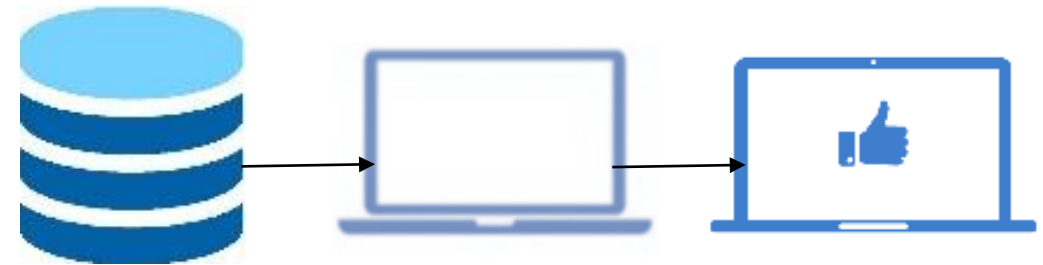
- **Illustration chiffrée de la qualité logicielle :**

- En 2020, les défauts de qualité logicielle aux Etats-Unis ont coûté **2 080 Md\$** à l'économie des États-Unis (source CISQ – voir [l'article du Monde Informatique](#)).
- Plus d'**1/3** aurait pu être économisé en appliquant des méthodes standardisées de recette.
- Pour développer les logiciels embarqués des satellites de la NASA, **80%** de la charge (en j.H) de développement est consacré aux tests. Pour l'informatique de gestion, la norme est de **35%**.



Le test :

- **Définition du dictionnaire :**
Épreuve, examen d'aptitude, essai.
- **Définition informatique / IEEE :**
Désigne une procédure de vérification d'un système.
- Le test examine une hypothèse en fonction de **3 éléments** :
 - Les données en entrée,
 - L'objet à tester,
 - Les observations attendues.
- **Rappel :**
Un **logiciel** prend des **données en entrée**, applique un **traitement** et fournit des **données en sortie**



Les questions que l'on se pose sur un logiciel à évaluer



- **Ce qui est produit répond-il à la demande ?**
 - Il doit se conformer aux spécifications ou aux normes contractuelles, légales ou réglementaires, ou aux attentes des utilisateurs
- **Comment vérifie-t-on qu'il est « conforme » ?**
 - En validant que l'objet de test est **complet et fonctionne comme attendu** par les utilisateurs finaux
 - En cherchant des **défaillances et défauts** afin qu'ils soient **corrigés** en phase de développement
- **Quel est l'objectif ?**
 - **Vérifier** si tous les besoins **spécifiés** ont été satisfaits
 - **Construire la confiance dans le niveau de qualité** de l'objet de test en **réduisant le niveau de risque**
 - **Fournir suffisamment d'information** aux parties prenantes pour **leur permettre de prendre des décisions éclairées**, en particulier en ce qui concerne le niveau de qualité de l'objet de test



Anomalie ou bug :

- **Définition informatique :**

Terme communément employé pour décrire une erreur, un défaut, un échec ou un manquement dans un programme informatique.



Définition ISTQB :

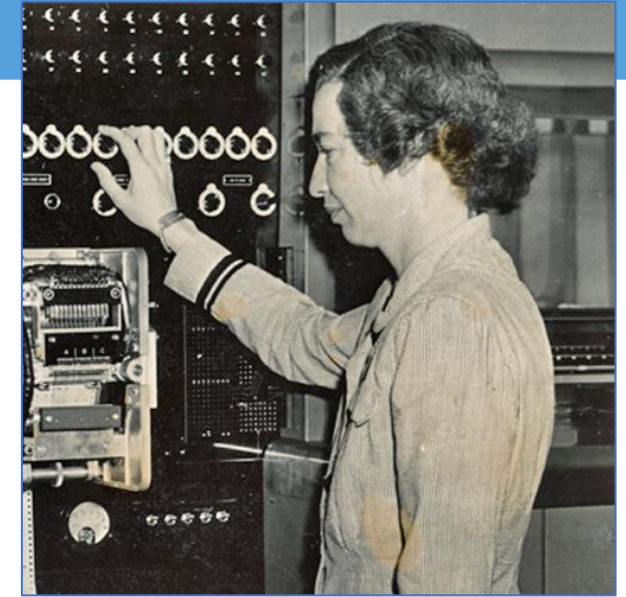
Toute **condition qui dévie des attentes** basées sur les exigences de spécifications, documents de conception, documents utilisateurs, standards etc., ou des perceptions ou expériences de quelqu'un.

Les anomalies peuvent être trouvées **pendant, mais pas uniquement**, les revues, tests, analyses, compilations ou utilisation des produits logiciels ou de la documentation applicable [IEEE 1044].



Anomalie : les origines

- **Définition du dictionnaire :**
Écart par rapport à la norme, irrégularité.
- **Bug (cafard) :** Vient du vocabulaire des ingénieurs de matériel électrique. Thomas Edison utilise déjà le terme en 1870.
- **« New Catechism of Electricity » de Nehemiah Hawkins (1896) :** « Le terme bug est employé pour désigner tout problème ou erreur dans le fonctionnement d'un appareil électrique. »
- **Anecdote de Grace Hopper :** En 1945, elle trouve un insecte (bug en anglais) coincé dans un contacteur électrique et a conservé le cadavre de l'insecte dans son journal de bord avec l'annotation « premier cas véritable de bug découvert ». Elle prit ensuite l'habitude de dire, quand elle recherchait la cause d'un problème, qu'elle « cherchait l'insecte » (« *looking for the bug* »).
- **NB :** [En savoir plus sur Grace Hopper](#)



9/9

0800 Antan started

1000 " stopped - antan ✓ { 1.2700 9.037 847 025
 1300 (032) MP - MC ~~1.982142000~~ 9.037 846 895 connect
 (033) PRO 2 2.130476415 ~~(23)~~ 4.615925059(-2)
 connect 2.130676415

Relays 6-2 in 033 failed special speed test
 in relay " 10.00 test.

Relays changed

1100 Started Cosine Tape (Sine check)

1525 Started Multi-Adder Test.

1545  Relay #70 Panel F
 (moth) in relay.

First actual case of bug being found.

~~1630~~ 1630 Antan started.

1700 closed down.

Relay 3145
 Relay 3370

- **Pourquoi trouve-t-on des anomalies ?**

- Selon [Frederick Brooks](#) : « *Il existe des bugs dans les logiciels parce que ce sont des logiciels.* »
- La présence de bugs n'est pas un accident mais elle est liée à la nature même des logiciels.
- Il n'existe pas de « [Silver Bullet](#) » (balle en argent) pour les éviter d'où la nécessité des tests.
- Il n'existe pas non plus de « [Golden Hammer](#) » permettant de les détecter à coup sûr *
- “*Testing can reveal the presence of errors but never their absence*”

[Edsger W. Dijkstra](#) in “Notes on structured programming”, ed. Academic Press, 1972.

* *Néanmoins, le folklore du Test évoque la légende de l'Ano maître, qui permettrait de les gouverner tous, l'Ano pour les trouver, pour les amener tous, et dans les ténèbres les lier...*

- Bug, anomalie, défaut... ? La chaîne causale :

Un être humain (ou un développeur informatique) peut commettre une **ERREUR**.



Cette erreur cause un **DÉFAUT** dans le code exécutable.



Quand le code est exécuté, le défaut peut produire une **DÉFAILLANCE**.



Cette défaillance se traduit par une **ANOMALIE**.

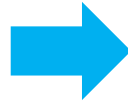
Le terme « bug » est trop générique !



L'artisan commet une **erreur** lors de la pose, au mois de juin.



Cette erreur entraîne un **défa**ut d'étanchéité. (Mais jusqu'ici, tout va bien, on est en été)



L'automne arrive, avec son lot de pluies et d'orages.



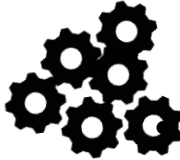
L'utilisateur constate la **défaillance**.

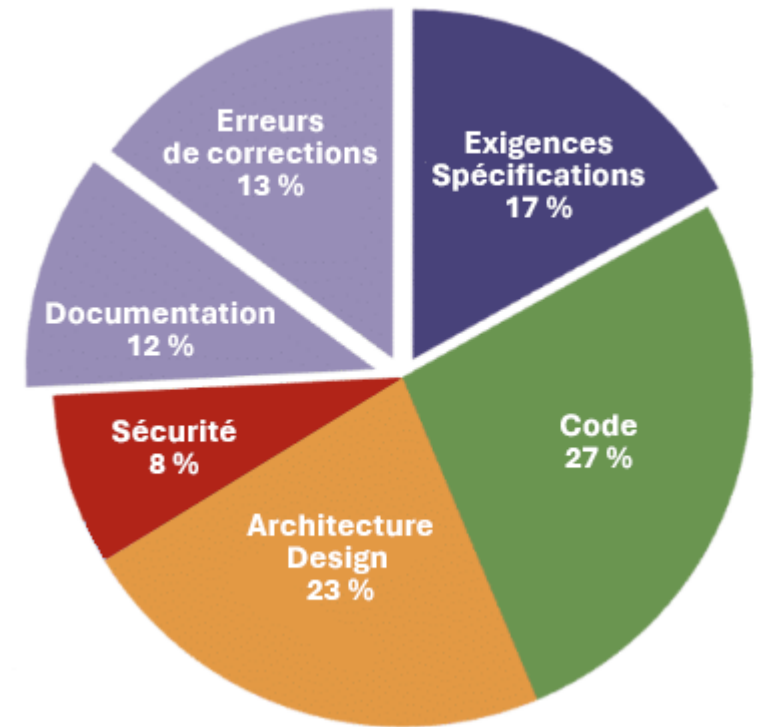


L'artisan, au nom de la garantie, effectue les **corrections** nécessaires.

Feriez-vous appel à nouveau à ce même artisan pour une nouvelle fenêtre ?

Les **erreurs** peuvent survenir pour de nombreuses raisons, telles que :

- Les contraintes de temps (ou de budget) 
- La faillibilité humaine 
- L'inexpérience ou le manque de compétence des participants au projet 
- Une mauvaise communication entre les participants au projet, y compris au sujet des exigences et de la conception 
- La complexité du code, de la conception, de l'architecture, du problème sous-jacent à résoudre, et/ou des technologies utilisées 
- Les malentendus sur les interfaces intra-système et inter-système 
- Des technologies nouvelles, peu connues 



Les causes racine des anomalies

(source : Caper Jones
« Applied Software Measurement »)

Test $\neq$ Débogage

Les **activités de tests** et **activité de débogage** sont **différentes**.

Les **testeurs**, en exécutant les **tests fonctionnels et non-fonctionnels** et en rapportant les dysfonctionnements mettent en évidence les **anomalies** (= défauts causés par une défaillance dans le code exécuté).

Le **débogage** est l'activité des **développeurs** qui **analysent, reproduisent et corrigent** les défauts **signalés**.

Par la suite, les **testeurs**, en exécutant les **tests de confirmation**, **vérifient** si les **corrections** apportées ont **résolu les défauts**.

-> Erreur

-> Défaut

-> Défaillance

-> **Anomalie**

-> **Débogage**

-> **Tests de confirmation**

-> **Tests de non-régression (TNR)**

Actions du développeur
Actions du testeur

1960 Naissance **des premières pratiques structurées de test**, notamment dans les domaines spatial et militaire, où la criticité des projets impose une fiabilité maximale. Le concept de « **génie logiciel** » gagne en visibilité (Conférence de l'OTAN, 1968).

Méthodes : premiers cycles de vie, langages de haut niveau et bonnes pratiques embryonnaires.

1970 L'idée de « **qualité logicielle** » prend forme : les développeurs pratiquent tests unitaires et tests d'intégration. Les **tests de non-régression (TNR)** se généralisent, consommant une part notable des ressources projets.

Méthodes : test unitaire, test d'intégration, TNR (non-régression).

1980 Le test devient une **activité spécialisée, souvent assurée par des développeurs dédiés**.

Les scénarios et les environnements de test se formalisent, tandis que les premiers outils automatisés apparaissent.

Méthodes : scénarios de test, outillage (scripts, plateformes), suivi informel.

1990 Montée en puissance de la qualité logicielle : normalisation et management de la qualité (**CMM, ISO 9001**). ISO/CEI 9126 définit un modèle de qualité logicielle. **CMM (Capability Maturity Model)** définit 5 niveaux de maturité (Initial, Repeatable, Defined, Managed, Optimizing)

Méthodes : Qualimétrie, performances, formalisation plus poussée (normes et référentiels).

2000 Le métier de testeur s'institutionnalise, avec des **rôles clairement identifiés et indépendants du développement**. Les premières certifications (ex. : ISTQB) renforcent cette professionnalisation. En 2002, CMMI (Capability Maturity Model Integration) étend la portée de CMM au-delà du développement.

Méthodes : compétences spécifiques, structuration dans l'organisation, séparation test/développement.

2010 Les testeurs deviennent **pluridisciplinaires**, s'inscrivant dans les approches Agile et DevOps. **ISO 25010** modernise le cadre qualité, l'automatisation du test se généralise et le métier de testeur occupe un rôle clé.

Méthodes : Agile, DevOps, tests continus, ISO 25010, certifications ISTQB.



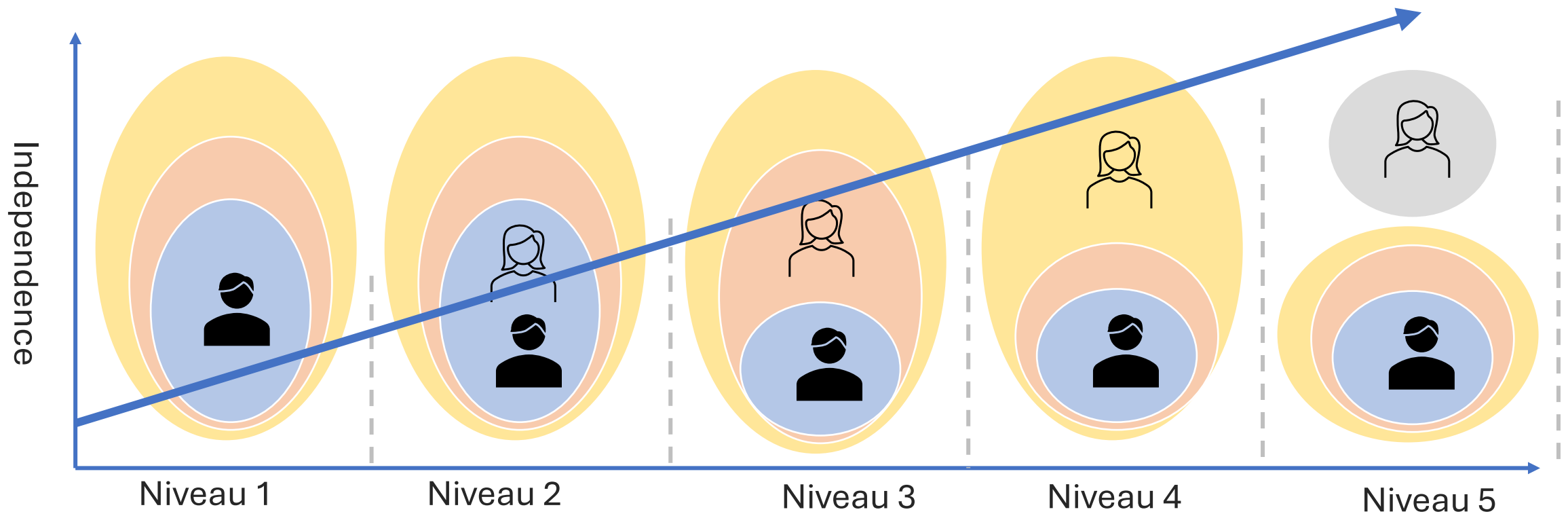
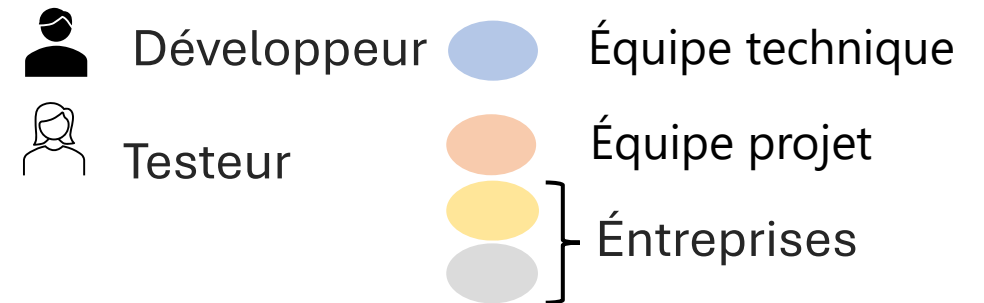
Un certain degré **d'indépendance** rend souvent le testeur plus **efficace** que les développeurs ou le métier pour détecter les **défauts** du fait des différences entre les **biais cognitifs** de l'auteur (le développeur) et du testeur.

Un élément de la psychologie humaine appelé **biais de confirmation** peut par exemple rendre difficile pour un individu d'accepter des informations en désaccord avec ses « croyances », ce qu'il est persuadé de savoir ou d'avoir compris (par exemple, les « terre-platistes » ...).

- *Suite à une livraison de la MOE, puisque les développeurs attendent de leur code qu'il soit correct, ils peuvent être victimes d'un biais de confirmation qui fait qu'il leur est difficile d'accepter que le code soit incorrect.*
- *La MOA, de son côté, peut avoir du mal à admettre que les SFD (Spécifications Fonctionnelles Détaillées) transmises aux développeurs comme aux testeurs comportent des lacunes, des inexactitudes, des incohérences voire des erreurs.*

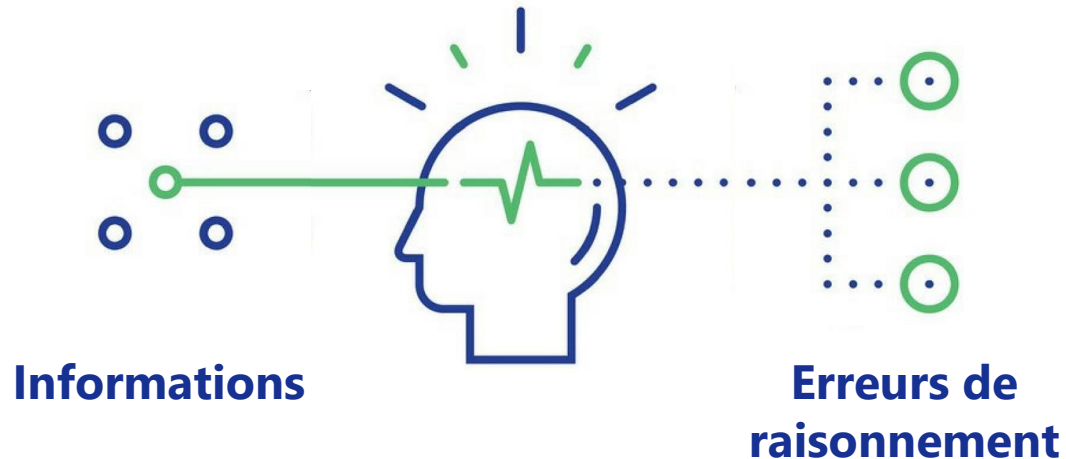
Par rapport aux développeurs, les **testeurs indépendants** sont donc plus susceptibles de détecter des types différents de défaillances, en raison de leurs connaissances propres, de leurs approches techniques et de biais différents.

Les degrés d'indépendance dans les tests comprennent les niveaux suivants (allant d'un niveau d'indépendance faible à un niveau élevé) :



Les biais cognitifs :

Définition : déviation de la pensée logique et rationnelle



Sur les biais cognitifs, voir aussi l'analyse de l'article de Salman, I. (2016), "Cognitive Biases in Software Quality and Testing", dans la version enrichie EQL du Syllabus ISTQB Foundation v.4

Les causes racine des biais cognitifs :

TROP D'INFORMATIONS

Donc mon cerveau se focalise sur...

- La répétition,
- Les trucs bizarres, drôles
- Les changements, les différences
- La confirmation

PAS ASSEZ DE SENS

Donc mon cerveau remplit les cases avec...

- La construction d'un sens
- Des généralités
- Des stéréotypes
- Ce que je connais déjà

NÉCESSITÉ D'AGIR VITE

Donc mon cerveau choisit

- Le plus près, le plus immédiat
- Le plus facile, simple, rapide
- Je reste sur ce que j'ai commencé
- Je me fais confiance

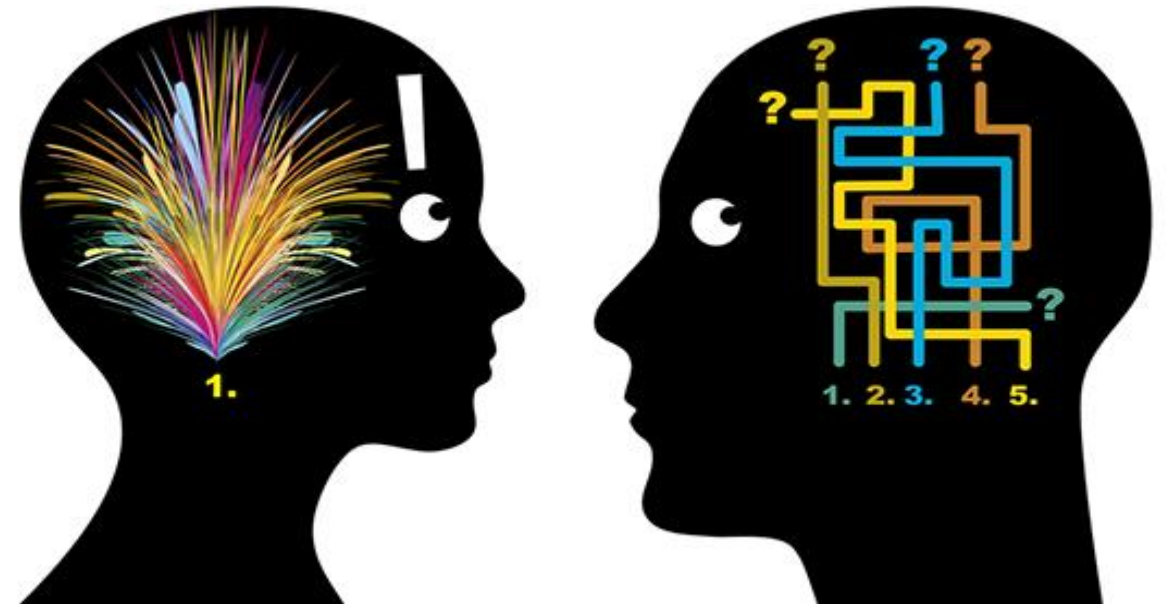
PAS ASSEZ DE MEMOIRE

Donc mon cerveau économise le stockage

- Je généralise
- Je compresse mes souvenirs
- Je mémorise le début et la fin
- Je garde 1 élément marquant

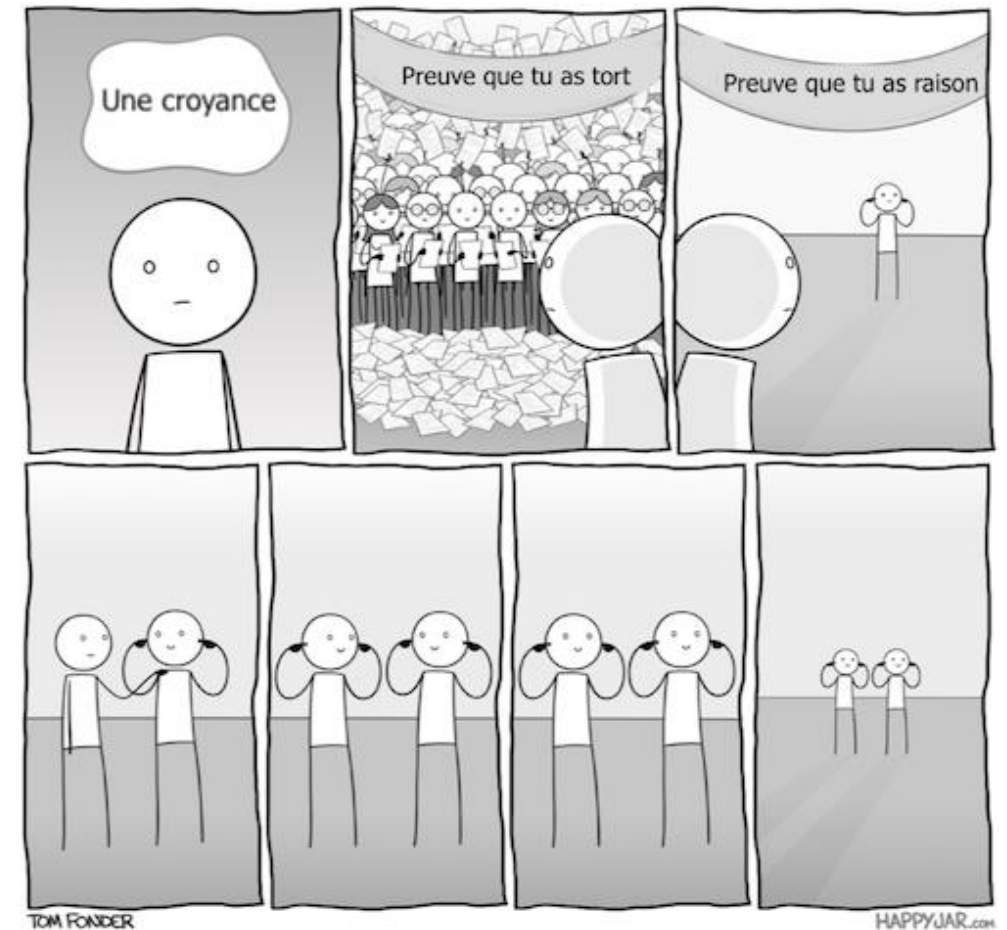
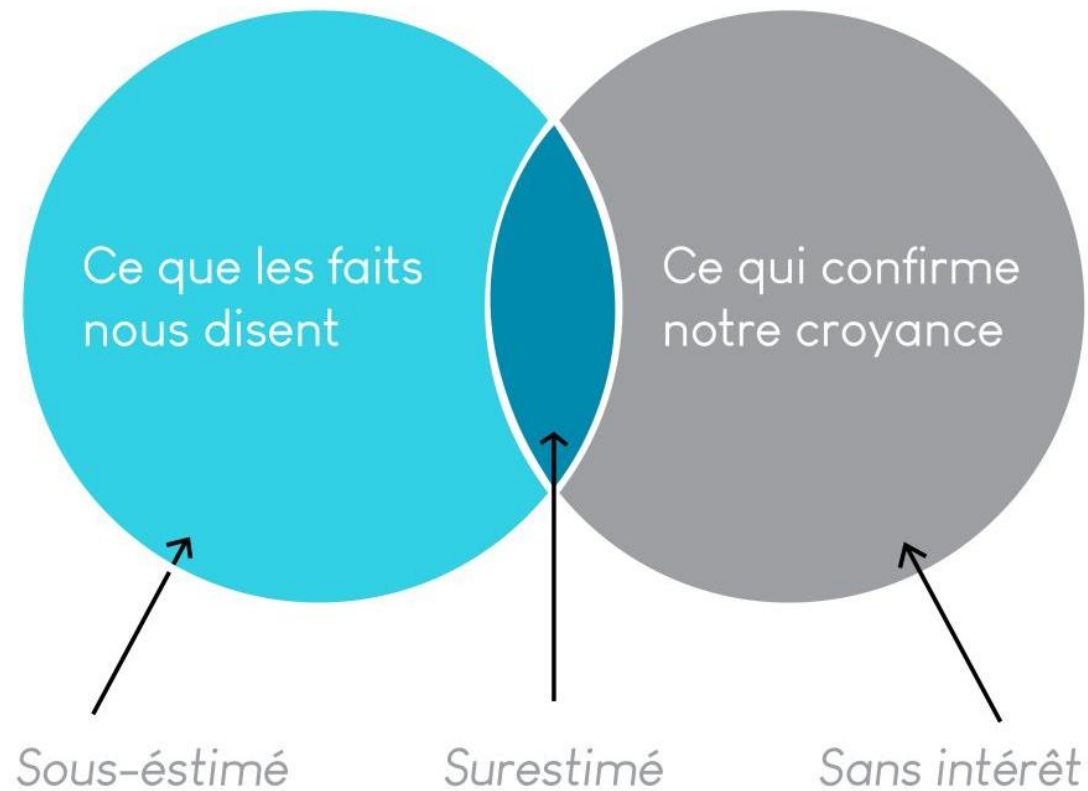
Psychologie des tests :

- Biais de confirmation
- Biais de congruence
- Erreur d'attribution fondamentale
- Biais du survivant
- Biais de négativité



- **Biais de confirmation :**

Consiste à ne considérer que les informations qui confirment ce que l'on croit et ignorer celles qui la contredisent.



- **Biais de congruence** : Consiste à accorder une confiance excessive à une hypothèse en ne testant que ce qui viendra la confirmer.



- Un testeur est **persuadé** qu'une fonctionnalité « X » est défectueuse.
- Pour « prouver » cette croyance, il ne conçoit que des **tests négatifs** ou des cas tordus susceptibles de provoquer une erreur, sans vérifier si, dans un usage normal ou dans d'autres conditions, la fonctionnalité pourrait fonctionner.
- Au final, il recueille facilement des cas où « X » échoue, mais il **passse à côté** de situations où « X » fonctionne correctement.
- La démarche est **symétrique** si l'on veut confirmer qu'« X fonctionne ».
- En test logiciel, si l'on veut **réellement** valider ou invalider une hypothèse (« ça marche » ou « ça ne marche pas »), il faut :
 - **Chercher des preuves de confirmation** (cas de test passants).
 - **Chercher activement des preuves de réfutation** (cas de test non-passants).
- Une couverture de test équilibrée devrait inclure les deux volets pour éviter le biais de congruence et obtenir un diagnostic objectif de la qualité logicielle.

- Erreur d'attribution fondamentale :

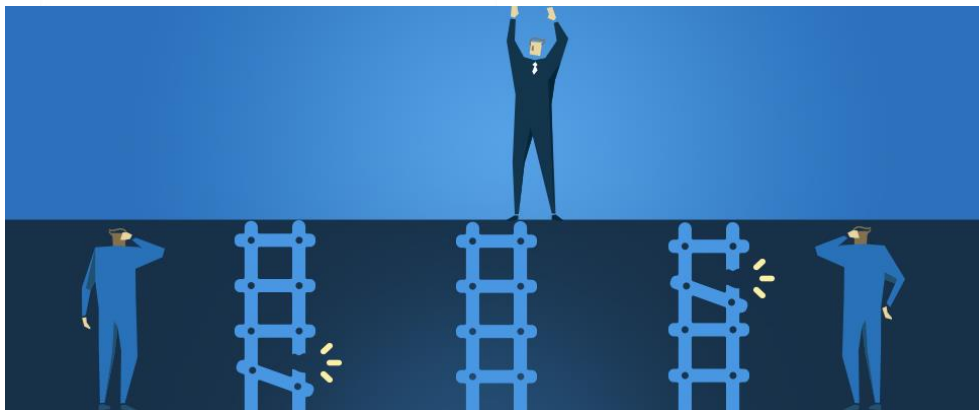
Interpréter des comportements en surestimant les causes internes et sous-estimant les causes externes.

Par exemple, imputer un dysfonctionnement du SUT à une erreur de codage du développeur alors que celui-ci relève d'une différence de l'environnement d'exécution ...



« **IL** » a grillé le stop. Il n'y avait personne, mais ça aurait pu !
C'est un chauffard.

« **JE** » grille un stop. Mais il n'y avait personne.
Je suis un conducteur prudent.



« **ILS** » n'arrivent pas à monter sur ce mur. Ils sont nuls !

« **MOI** » je l'ai fait facilement, I'm the Best !

- **Biais du survivant** : Notre cerveau ne remarque pas les preuves silencieuses !

Concentrer sa vision sur ce qui a réussi (au moins une fois), sans tenir compte de toutes les fois où ça n'a PAS marché.

« Une décision stupide qui fonctionne une fois devient une décision brillante rétrospectivement ». (Daniel Kahneman)

Donc, la prochaine fois qu'on sera confronté à un problème comparable, on adoptera la même solution ...

LES BIAIS COGNITIFS: LE BIAIS DU SURVIVANT

J'AI 80 ANS, JE
SUIS EN PLEINE
FORME ET JE
FUME DEPUIS 65
ANS!!!

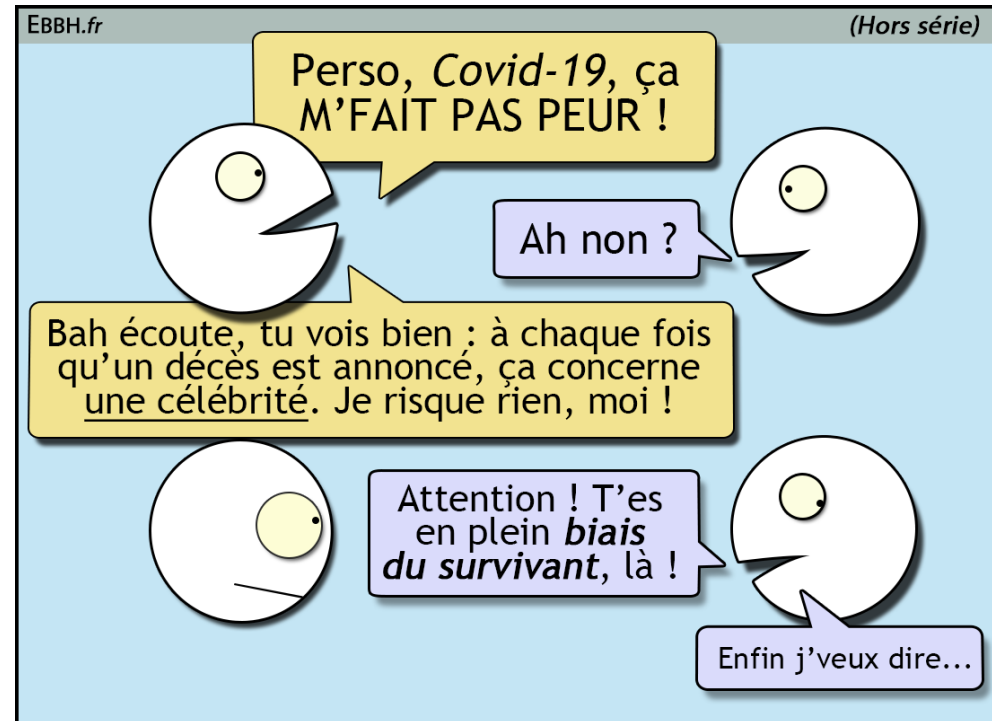
UN BON CONSEIL,
SI VOUS VOULEZ
VIVRE LONGTEMPS,
FAITES COMME MOI:
20 À 30 CIGARETTES
PAR JOUR, ENTRE
LES REPAS...



ÇA DEMANDE UN
PEU DE DISCIPLINE,
MAIS JE SUIS
LA PREUVE VIVANTE
QUE ÇA MARCHE!

COMIC SCIENCE.NET

JEAN-MICHEL ÇAMARCHEPOURMOI. TABACOPATHE DEPUIS PLUS D'UN DEMI SIÈCLE

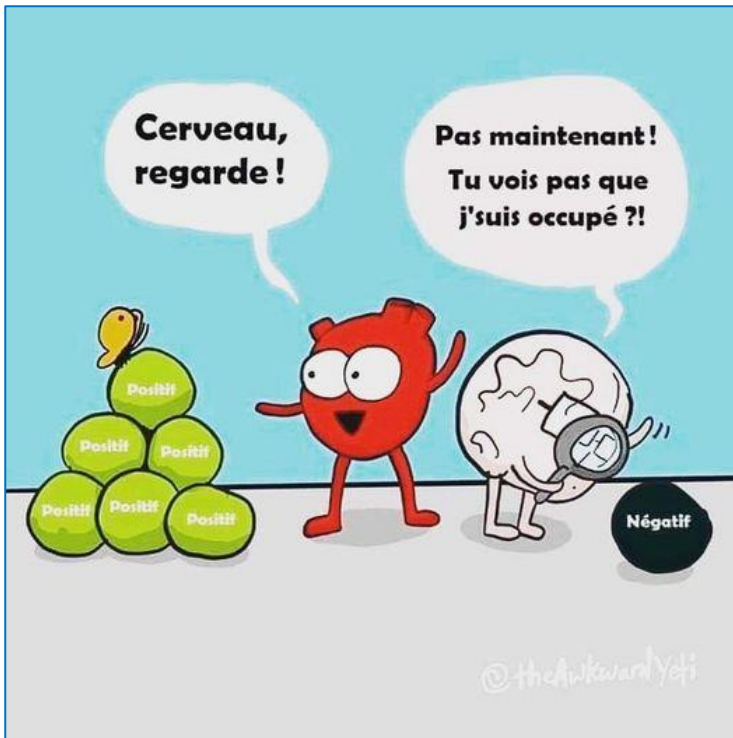


- **Biais de négativité :**

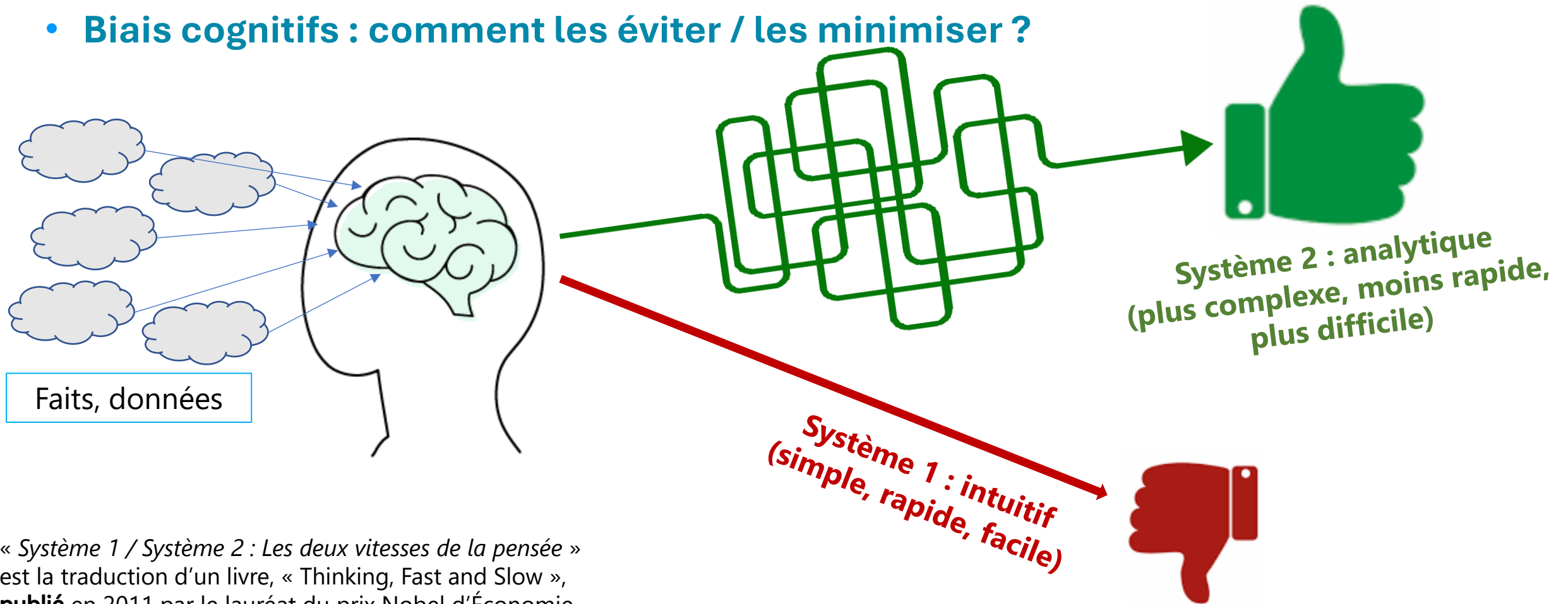
Surestimer les aspects négatifs d'une situation

« *Un arbre qui tombe fait plus de bruit qu'une forêt qui pousse* »

Si on expose uniquement ce qui « ne marche » pas, et pas « ce qui marche », quelle vision aura votre auditeur ?



- **Biais cognitifs : comment les éviter / les minimiser ?**



« Système 1 / Système 2 : Les deux vitesses de la pensée » est la traduction d'un livre, « Thinking, Fast and Slow », **publié** en 2011 par le lauréat du prix Nobel d'Économie Daniel Kahneman. Découvrez ([lien Wikipedia](#))

En savoir plus : Exercez votre esprit critique ([Openclassrooms](#))

1. Les tests montrent la présence de défauts, pas leur absence

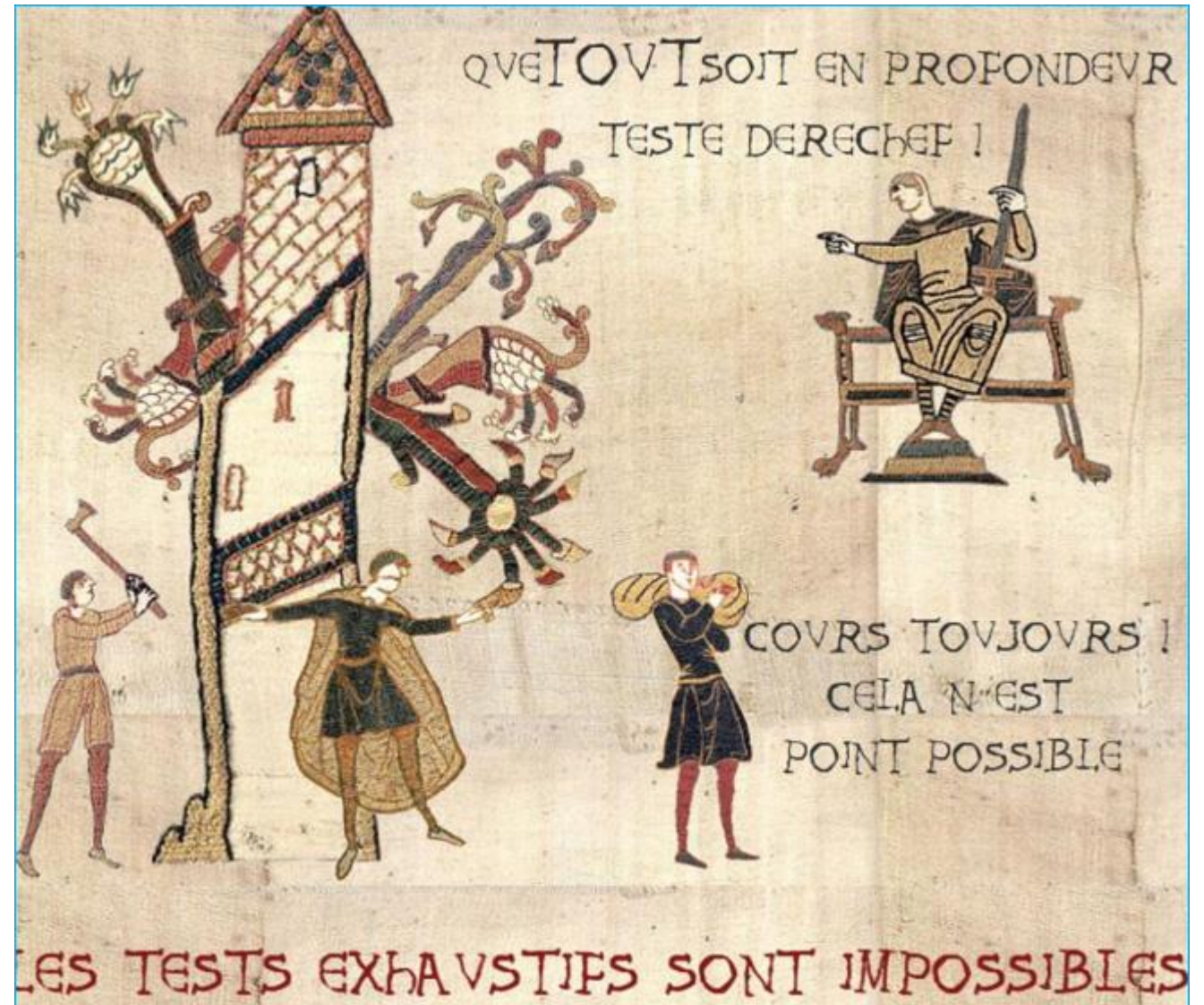
Les tests réduisent la **probabilité** que des défauts restent cachés dans le logiciel mais, même si aucun défaut n'est découvert, **ce n'est pas une preuve que tout est correct.**



2. Les tests exhaustifs sont impossibles

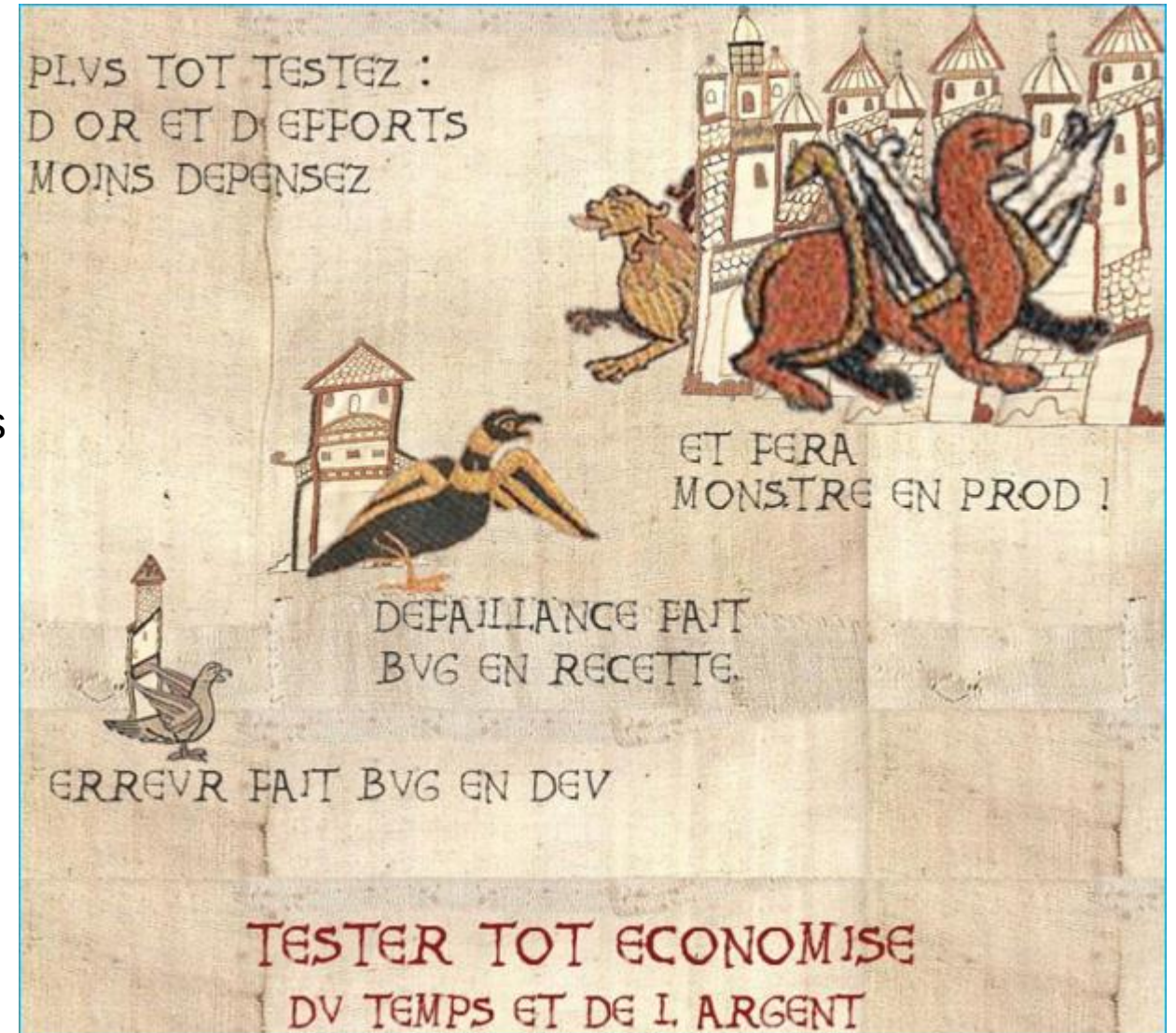
Tout tester (toutes les combinaisons d'entrées et de préconditions) **n'est pas faisable** sauf pour des cas triviaux.

Plutôt que de prévoir des tests exhaustifs, l'analyse des risques, des techniques de test et des priorités devraient être utilisés pour **cibler les efforts de tests**.



3. Tester tôt économise du temps et de l'argent

Pour **détecter tôt les défauts**, à la fois des activités de tests statiques et des activités de tests dynamiques doivent être **lancées le plus tôt possible** dans le cycle de vie de développement du logiciel. Tester tôt dans le cycle de vie du développement logiciel permet **de réduire ou d'éliminer des changements coûteux**.



4. Regroupement des défauts

Un **petit nombre de modules** contient généralement **la plupart des défauts découverts** lors des tests avant livraison, ou est responsable de la plupart des défaillances en exploitation.

Des **regroupements prévisibles de défauts**, ou **des regroupements réellement observés** en test ou en exploitation, constituent un élément important de **l'analyse des risques** utilisée pour **cibler l'effort de test** (comme mentionné dans le principe 2).



5. Paradoxe du pesticide

Si **les mêmes tests** sont **répétés de nombreuses fois**, le même ensemble de cas de tests finira par **ne plus détecter de nouveaux défauts**.

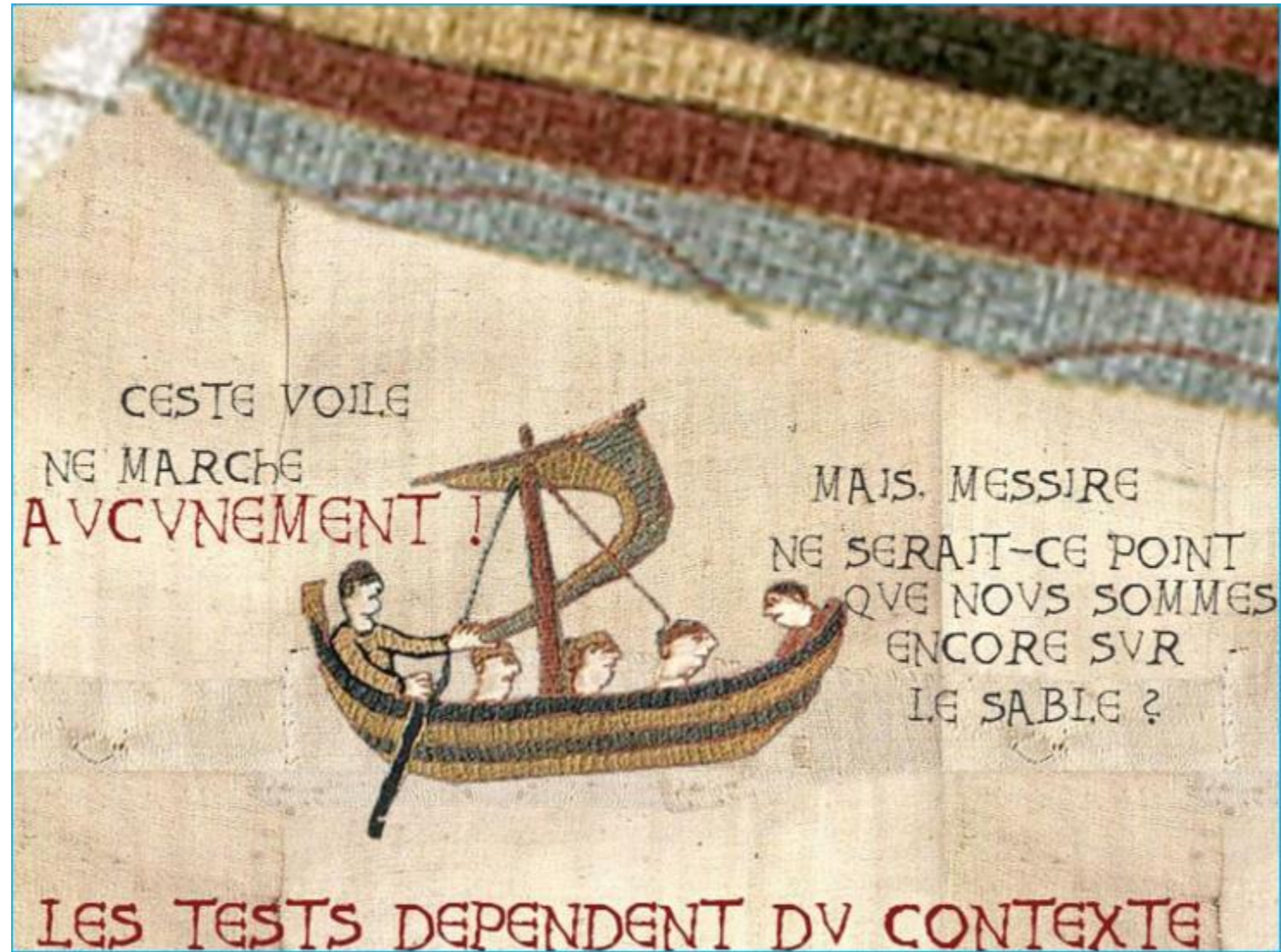
Pour détecter de nouveaux défauts, il peut être nécessaire de **modifier les tests existants** et les **données de test existantes**, ainsi que de **rédiger de nouveaux tests**.



6. Les tests dépendent du contexte

Les tests sont effectués différemment dans des **contextes différents**. Par exemple, un logiciel de contrôle industriel, critique au niveau de la sûreté, sera testé différemment d'une application de commerce électronique sur téléphone mobile.

De même, le test dans un **projet Agile** est effectué **différemment** du test dans un projet à **cycle de vie séquentiel**.



7. L'absence d'erreurs est une illusion

Certaines organisations s'attendent à ce que les testeurs puissent **effectuer tous les tests possibles** et **trouver tous les défauts possibles**, mais **c'est impossible**. De plus, il est **illusoire** (c.-à-d. une croyance erronée) de s'attendre à ce que **le simple fait de trouver et de corriger un grand nombre de défauts garantisse la réussite d'un système**.

Par exemple, tester en profondeur toutes les exigences spécifiées et **corriger tous les défauts trouvés pourrait toujours produire un système difficile à utiliser**, ne répondant pas aux besoins et attentes des utilisateurs ou moins performant comparé à d'autres systèmes concurrents.

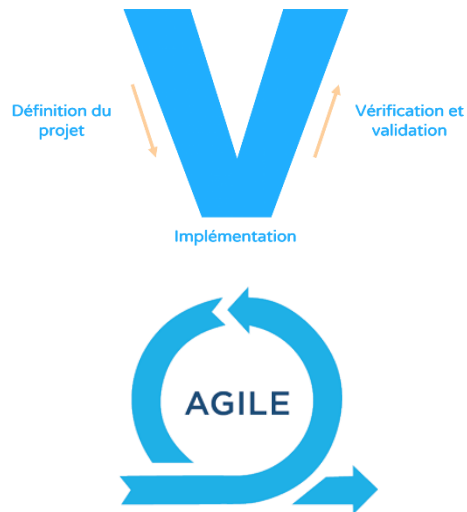


3. Cycle de vie du logiciel

Le « cycle de vie d'un logiciel » désigne toutes **les étapes du développement** d'un logiciel.

L'objectif du découpage est de permettre de définir des **jalons intermédiaires** : les étapes du projet et les documents à produire permettant la validation de chacune des étapes.

Selon les organisations et/ou selon les projets les cycles de développement adoptent différentes méthodes.

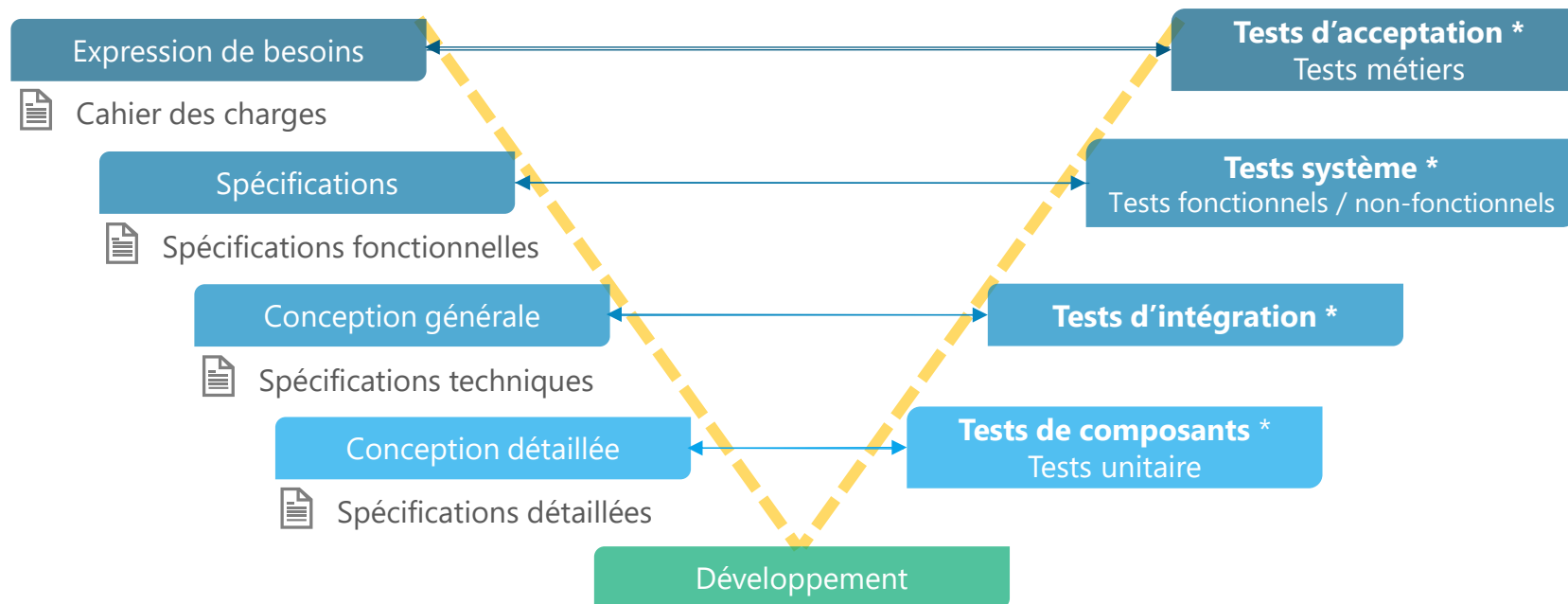


- La méthode séquentielle la plus répandue consiste à d'abord définir les besoins (MOA), développer le logiciel (MOE), tester le logiciel (QL) puis déployer l'outil fini en production.
C'est la méthode, dite **cycle en V**, efficace car bien cadrée successivement mais rigide et laissant peu de place aux modifications en cours de route.
- Les **méthodes Agile**, itératives et incrémentales, cadrées en 2001 via le [Manifeste Agile](#), offrent plus de souplesse pour incrémenter au cours du projet des ajustements selon le retour du client.
Les équipes agiles sont pluridisciplinaires et réalisent des incréments successives produites par cycles courts qui s'enchaînent, appelés « sprints ».

Modèle de développement séquentiel :

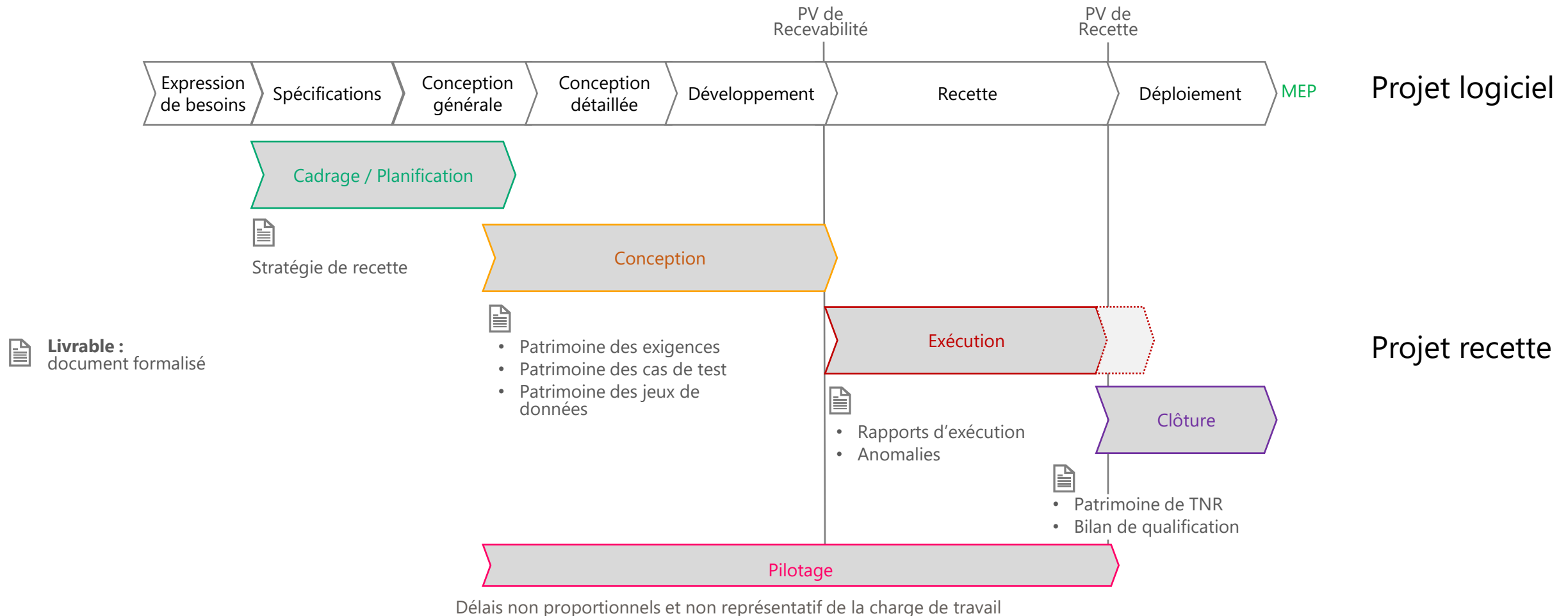
Dans la majorité des projets informatiques, l'approche du cycle en V reste privilégiée.

Chaque phase doit être terminée et validée avant que la phase suivante puisse démarrer.



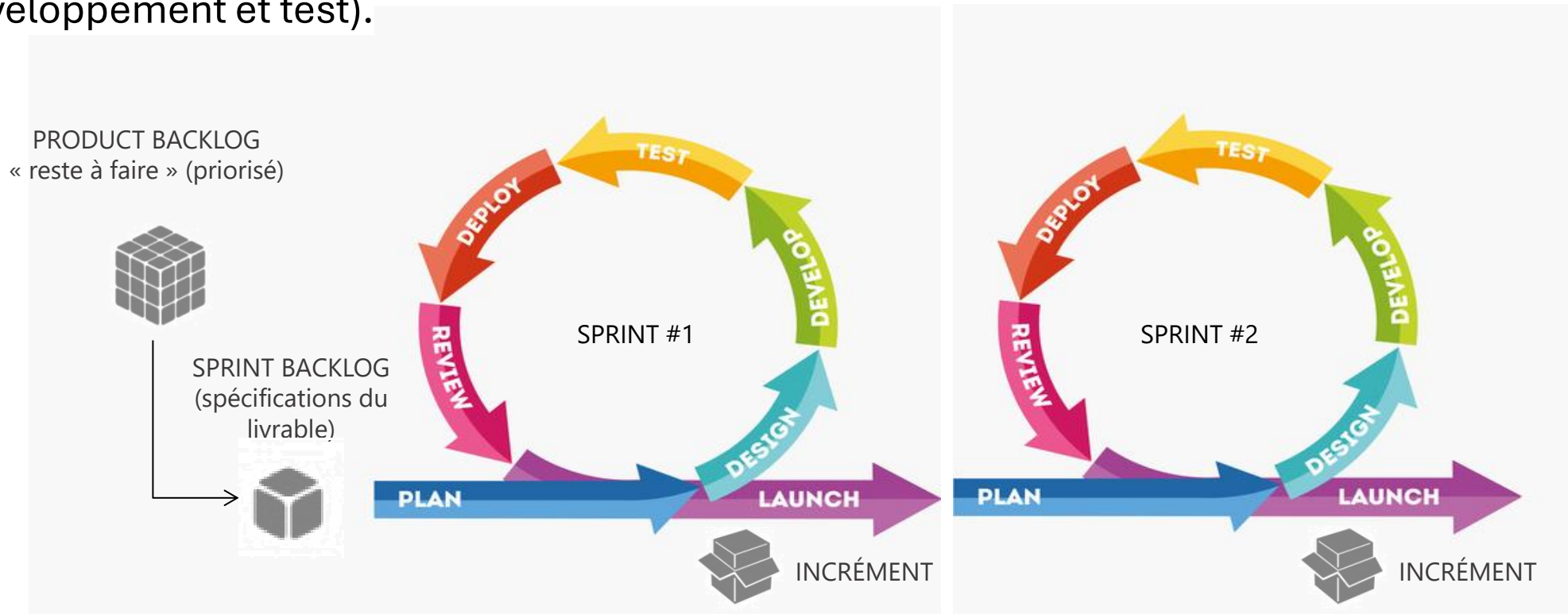
* Niveaux de test ISTQB

La qualification du projet est exécutée en **fin de projet**, mais sa **préparation** commence dès la phase de spécifications, en **amont du projet**.

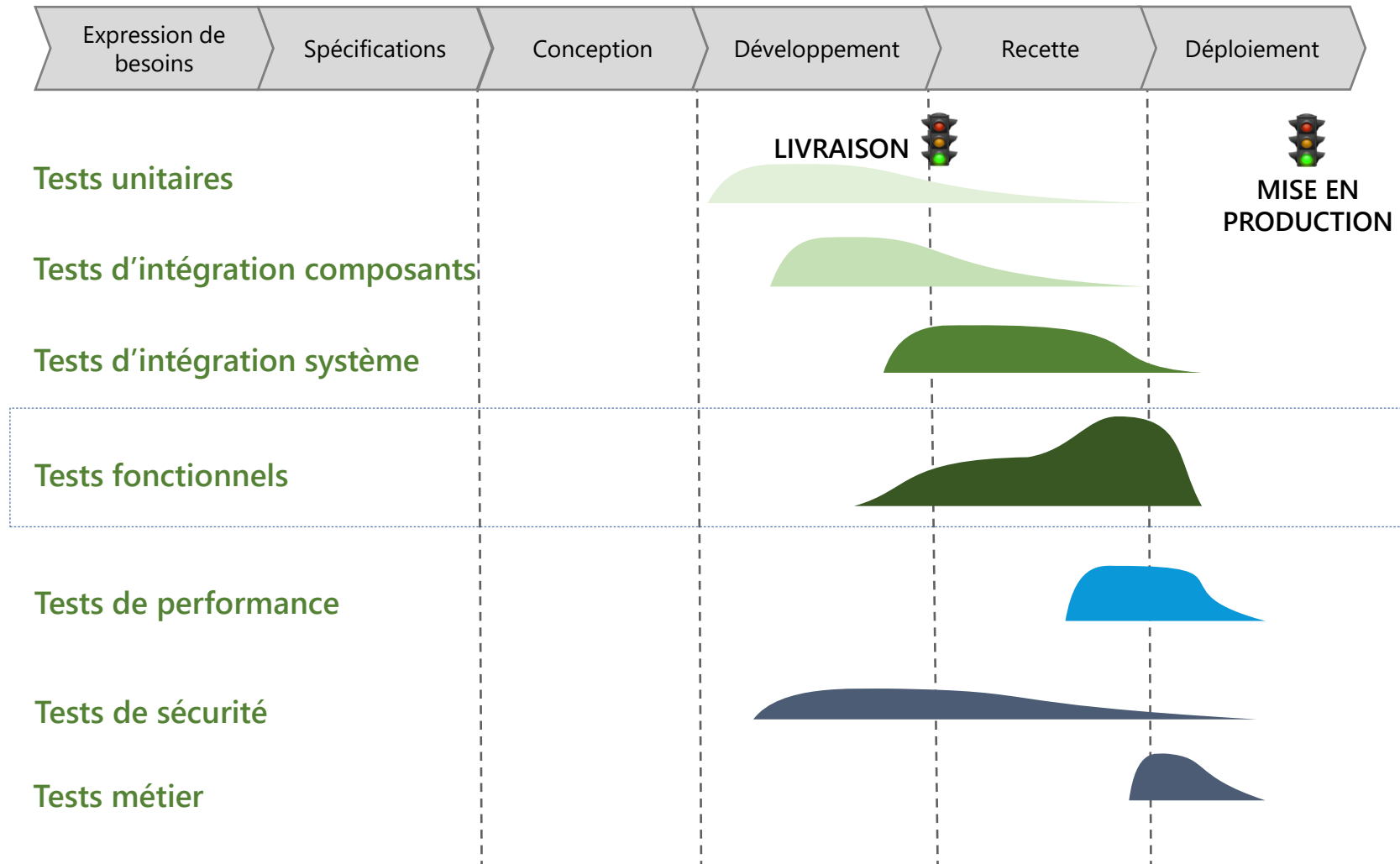


Modèle de développement itératif et incrémental :

Les méthodes **Agiles**, dont la méthode Scrum, s'appuient sur un **développement incrémental** de l'applicatif, où l'on va retrouver les principales étapes du cycle de développement (spécifications, conception, développement et test).



A chaque phase du projet informatique, correspond un **type de test pertinent**, impliquant différents niveaux de **charge**.



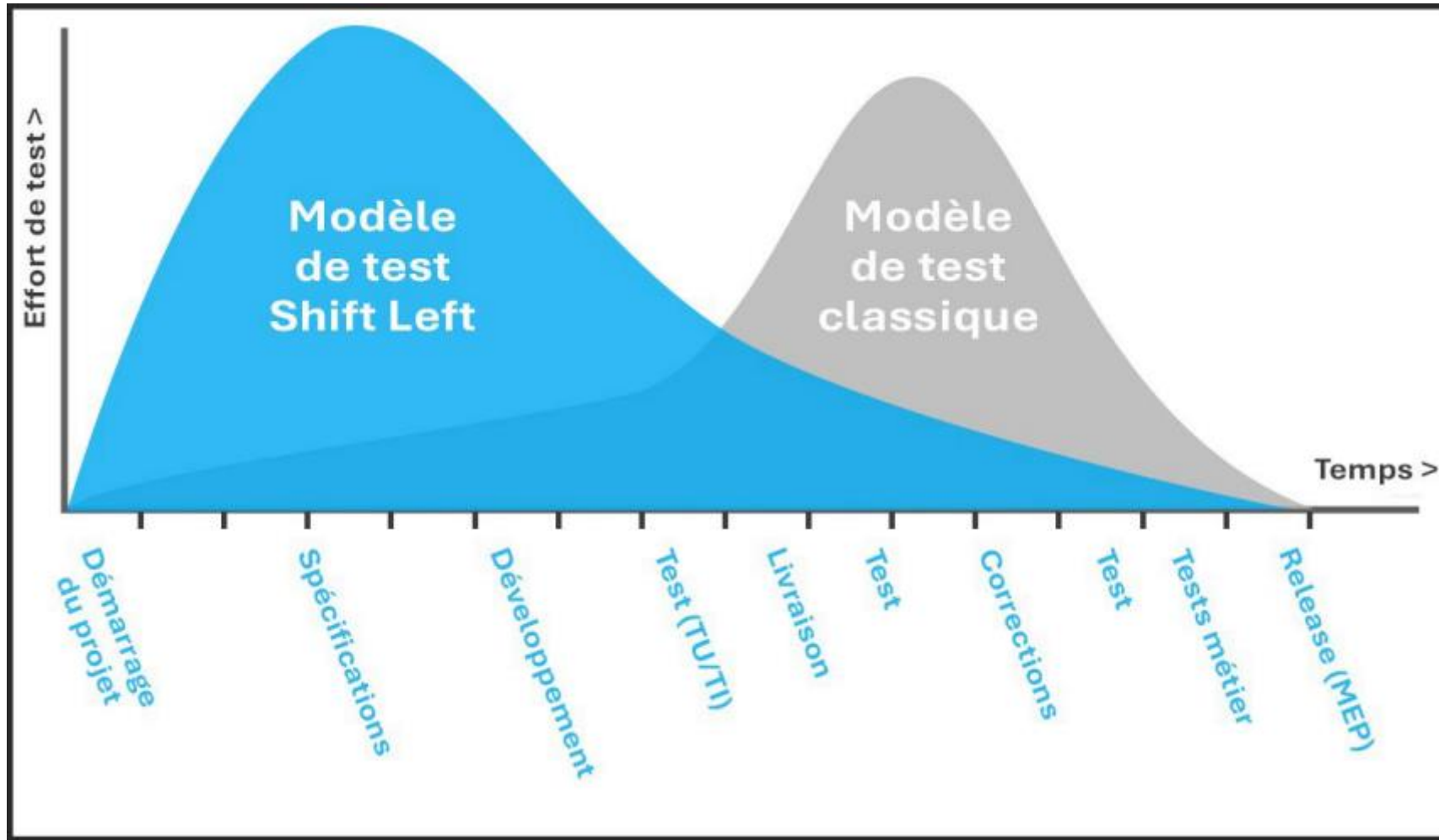


- Définition ISTQB du Shift Left :

Le **principe du test précoce**, parfois appelé **shift left**, est une approche dans laquelle le test est effectué **plus tôt** dans le cycle de vie du développement logiciel (avant implémentation du code ou l'intégration des composants par exemple).

Une approche shift left peut entraîner des **formations**, des **efforts** et/ou des **coûts supplémentaires au début du processus**, mais devrait permettre d'**économiser** des efforts et/ou des coûts **à un stade ultérieur** du processus.

Pour l'approche shift left, il est important que les **parties prenantes** soient **convaincues** et **adhèrent** à ce concept.



Les tests commencent **plus tôt**, mais **restent importants** dans **toutes les phases** du cycle de vie du développement logiciel.

4. Processus de test



« Nous avons réalisé des tests utilisateurs en 1984, 5 ans avant tout le monde. [...] Il y a une grande différence entre les faire, les laisser faire par les gens du marketing et les laisser faire par les ingénieurs qui ont été impliqués au cœur du projet. »

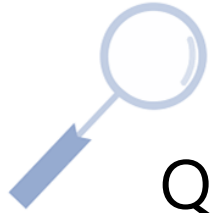
Scott Cook, co-fondateur de Intuit Inc.



- Définitions ISTQB des types de tests :
 - **Test dynamique** : Test *nécessitant l'exécution* du logiciel testé
 - *Vérifier la connexion au site*
 - *Remplir un formulaire*
 - *Naviguer entre les pages*
 - ...
 - **Test statique** : Test *ne nécessitant pas l'exécution* du logiciel testé
 - *Analyser la documentation*
 - *Analyser le code*
 - ...



Quel type de test est spécifique au testeur fonctionnel ?



Quel type de test est spécifique au testeur fonctionnel ?

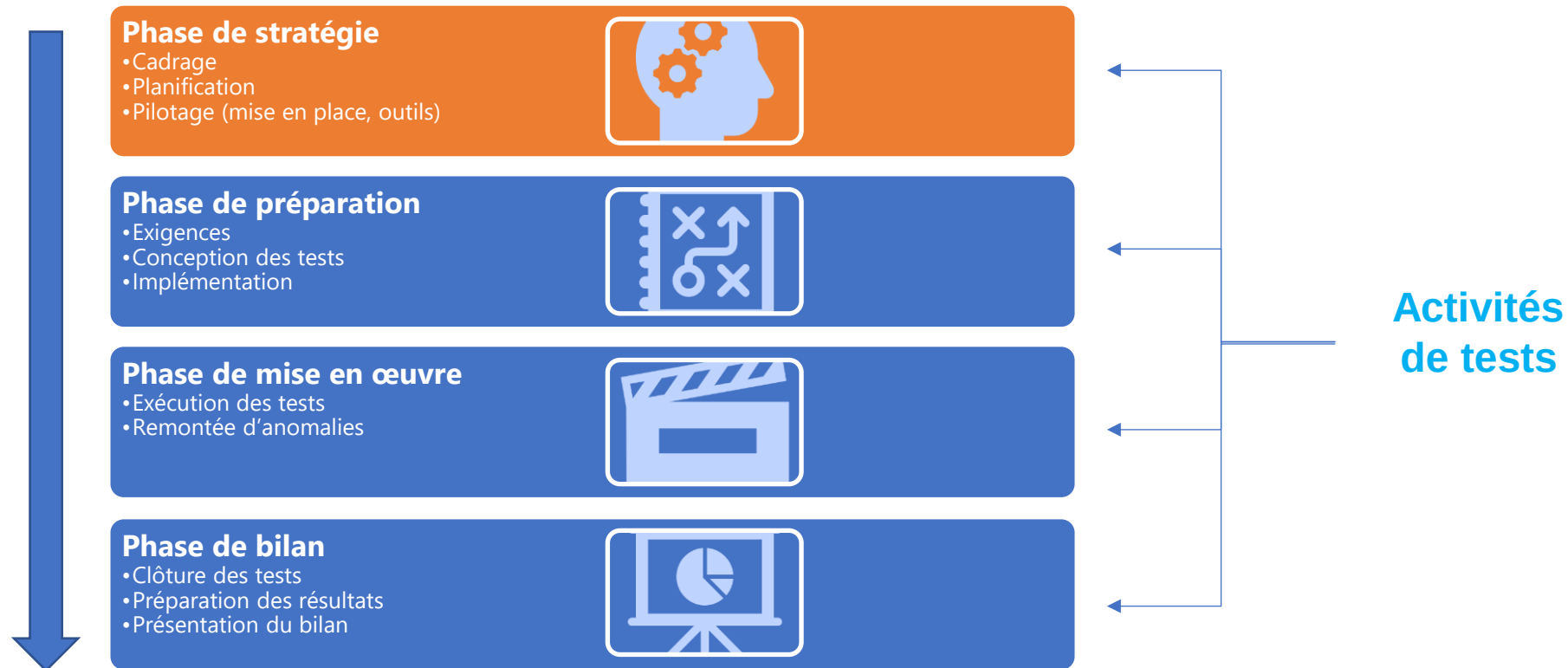
→ Beaucoup de tests dynamiques lors de la phase d'exécution, MAIS aussi des tests statiques, notamment en phase de Conception.

Un type de test statique très souvent réalisé est la [revue](#).

Toutes les **activités de test** s'inscrivent dans ce qu'on appelle un « **processus de test** »
S'il n'existe **pas** de **processus de test** logiciel « **universel** », il existe des ensembles **d'activités de tests** permettant d'atteindre les objectifs fixés.

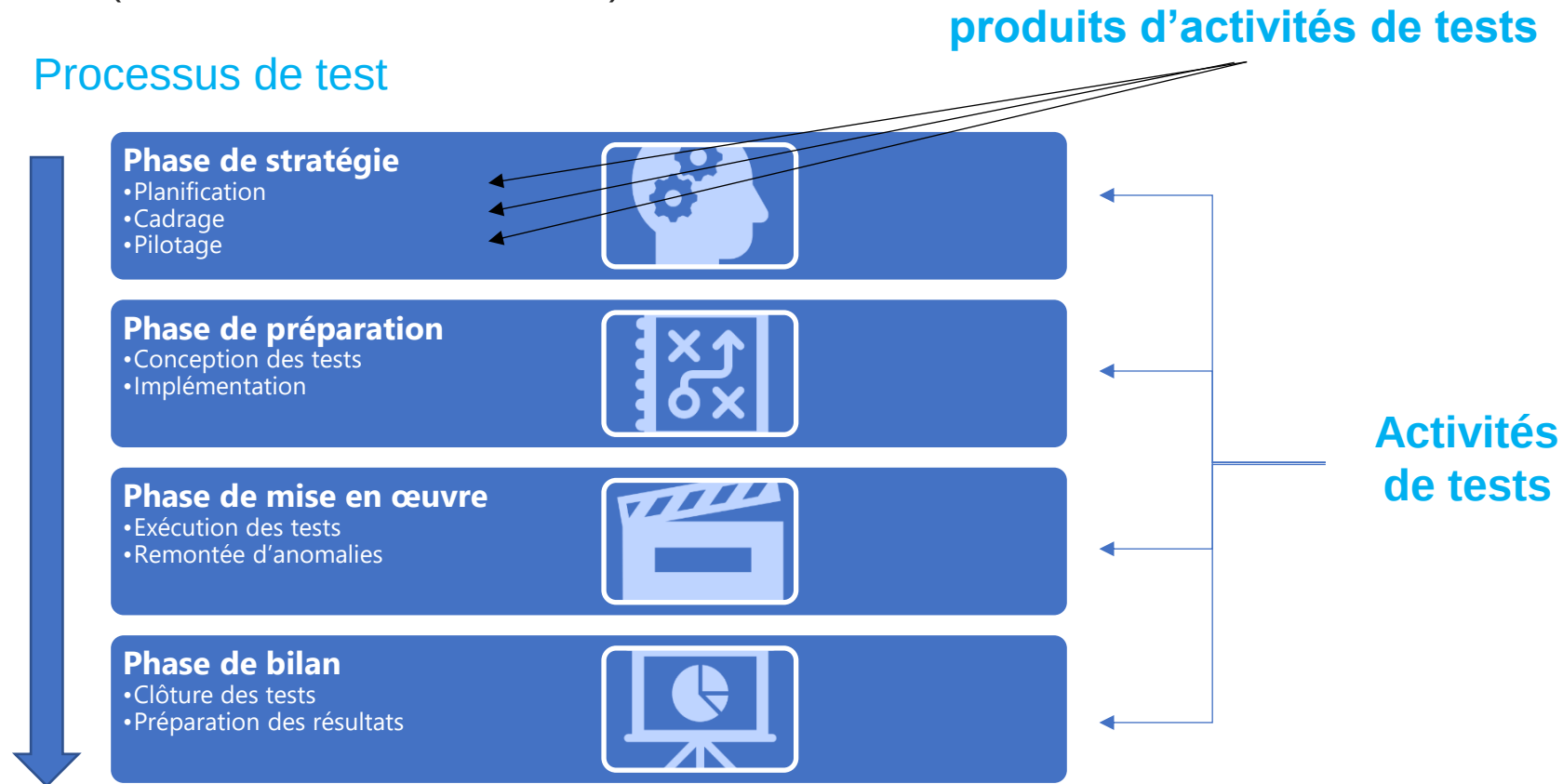
Ces activités de tests peuvent être regroupés en **différentes phases** :

Processus de test

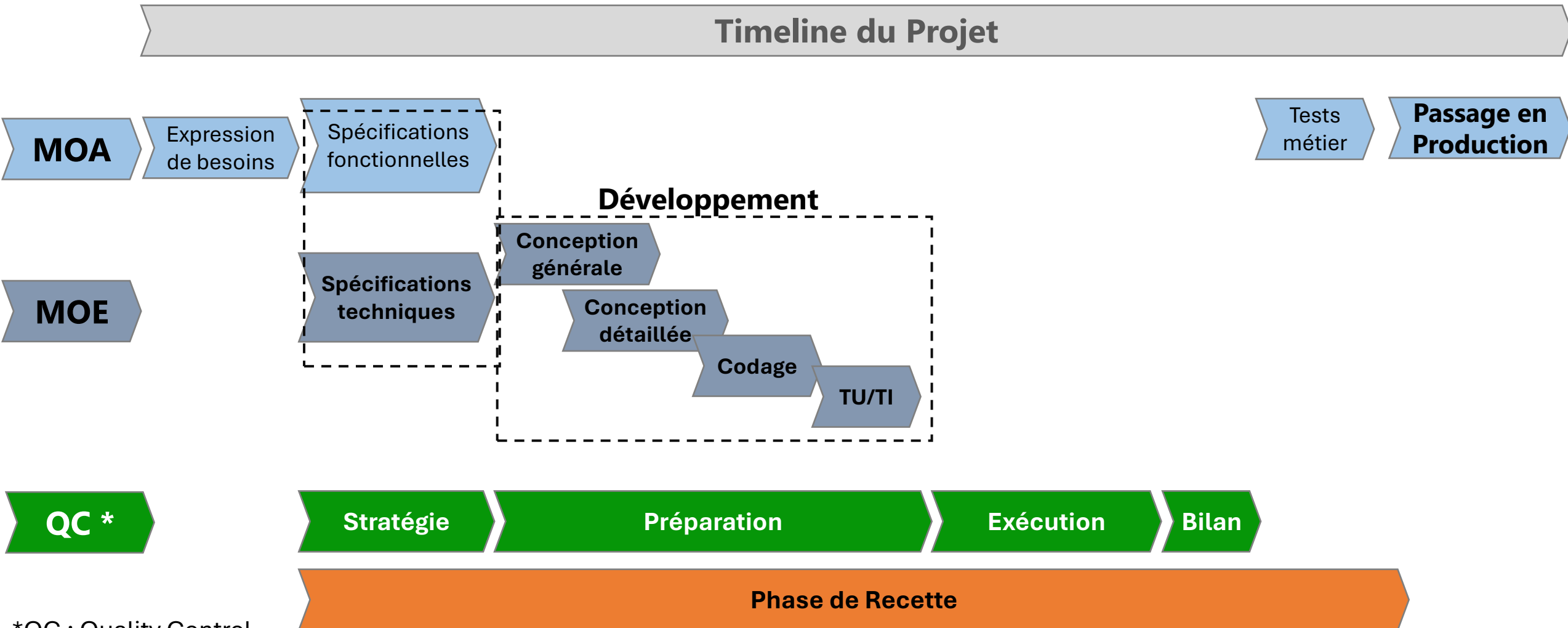


Dans le cadre d'un **processus de test** sont menées des **activités de test**, et créés des **produits d'activités de test**

Ces produits sont saisis et gérés dans des **outils de gestion de test** et des **outils de gestion de défauts** (Activités de test outillées).



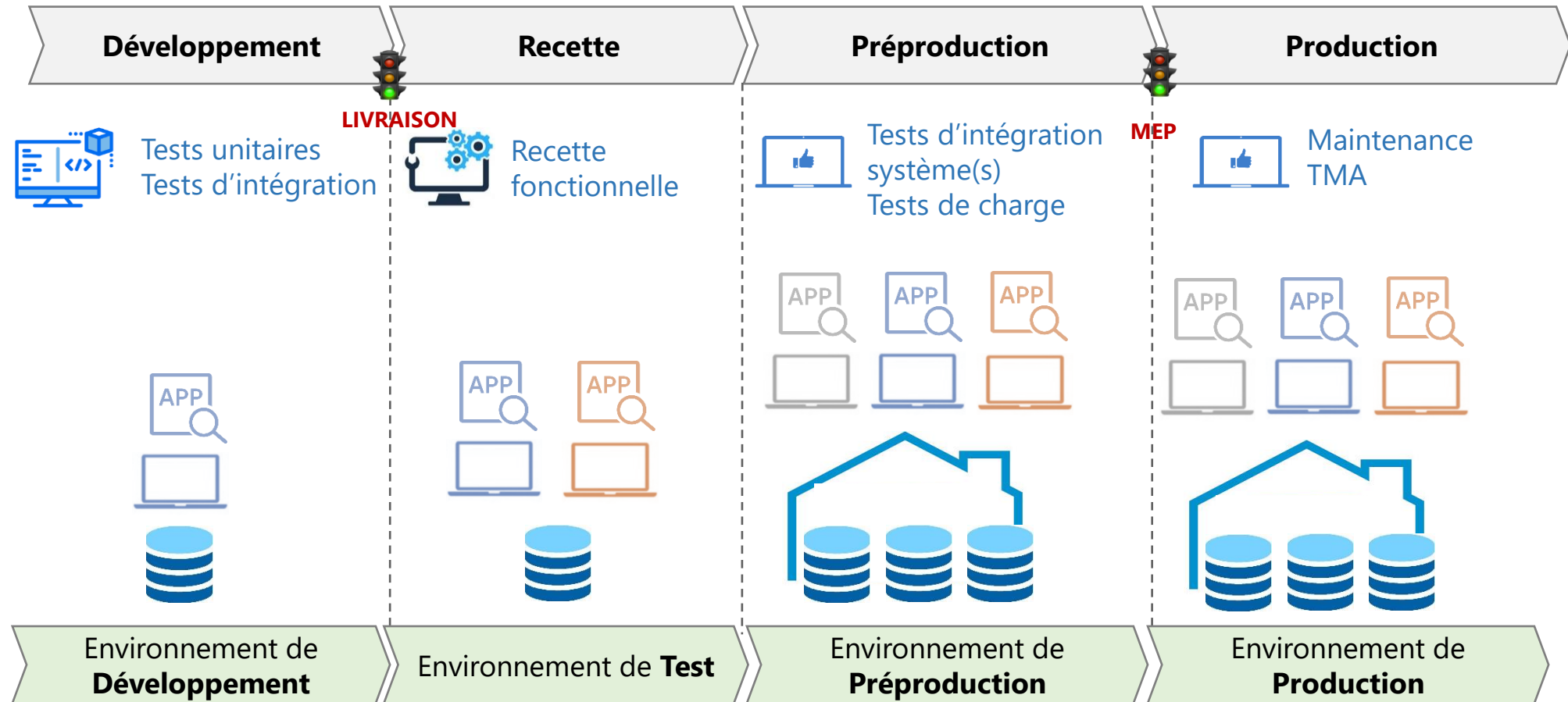
- Zoom sur les phases de la recette fonctionnelle



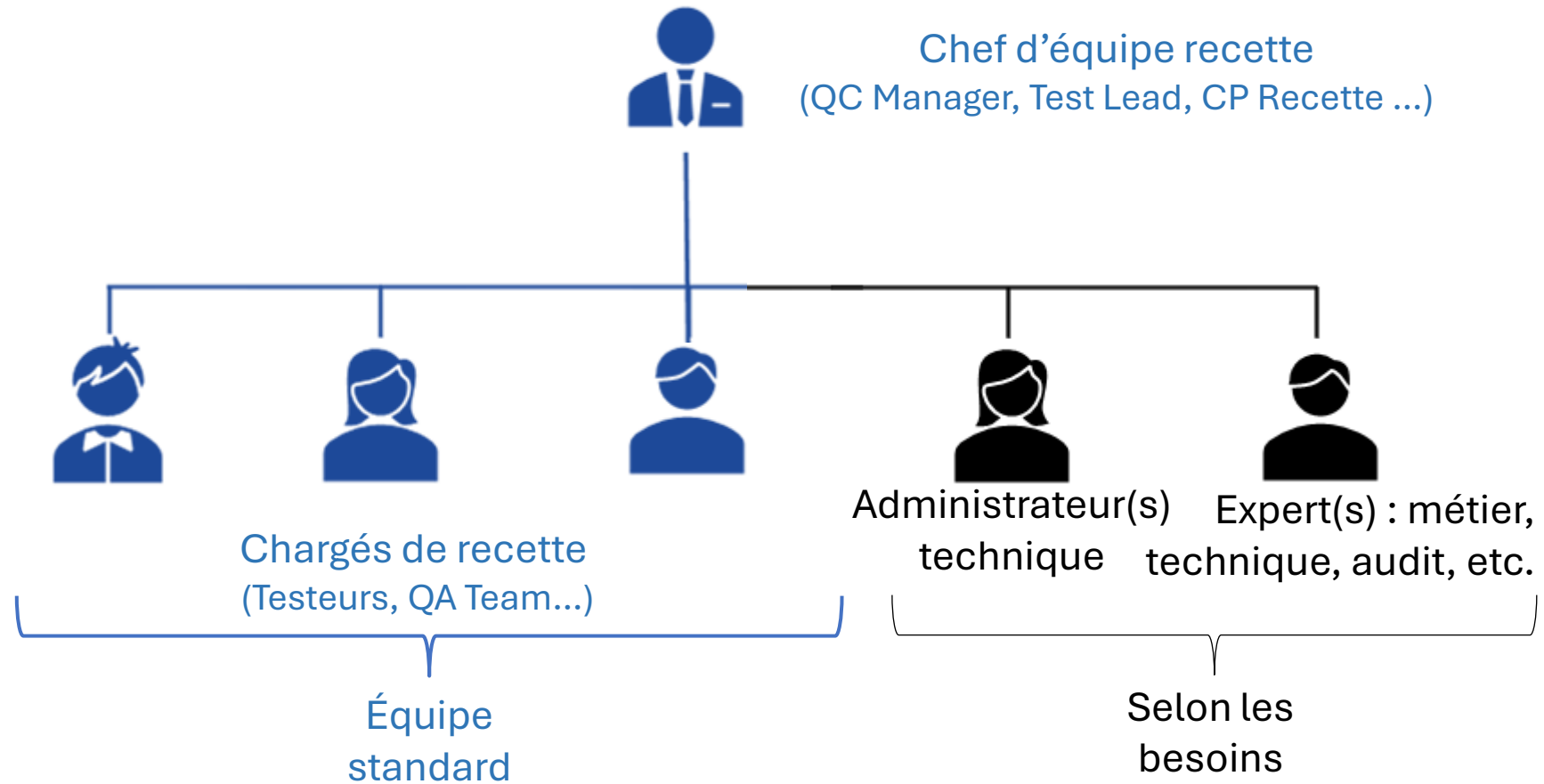
*QC : Quality Control

Les différentes activités de test **ne doivent pas impacter le SI opérationnel**. Elles se dérouleront dans des **environnements dédiés**, de plus en plus « réalistes » et « iso-prod » (semblables à l'environnement de production).

ON NE TESTE PAS AVEC LES DONNÉES DE PRODUCTION SUR L'ENVIRONNEMENT DE PRODUCTION !



- Les acteurs de la recette :





- Définitions ISTQB de l'équipe intégrée :

L'approche **équipe intégrée** repose sur la capacité d'un testeur à **travailler efficacement dans un contexte d'équipe** et à **contribuer positivement aux objectifs** de l'équipe.

Dans l'approche intégrée, **tout membre de l'équipe disposant des connaissances et des compétences nécessaires peut effectuer n'importe quelle tâche**, et **chacun est responsable de la qualité**.

Caractéristiques d'une équipe intégrée :

- Les membres de l'équipe **partagent le même espace de travail** (physique ou virtuel), car le **regroupement facilite la communication et l'interaction**.
- Les testeurs travaillent **en étroite collaboration** avec les autres membres de l'équipe afin de garantir que les niveaux de qualité souhaités sont atteints.
 - Ils travaillent **avec les représentants métier** pour les aider à **créer des tests d'acceptation adaptés**.
 - Ils travaillent **avec les développeurs** pour convenir de la **stratégie de test** et décider des **approches d'automatisation** des tests.
- Les testeurs **transfèrent leurs connaissances** en matière de test aux **autres membres** de l'équipe et **influencent le développement** du produit.

L'approche équipe intégrée :

- Améliore la **dynamique d'équipe**
- Améliore la **communication**
- Améliore la **collaboration** au sein de l'équipe
- Crée une **synergie** en permettant aux **différents ensembles de compétences** au sein de l'équipe d'être **exploités** au profit du **projet**.



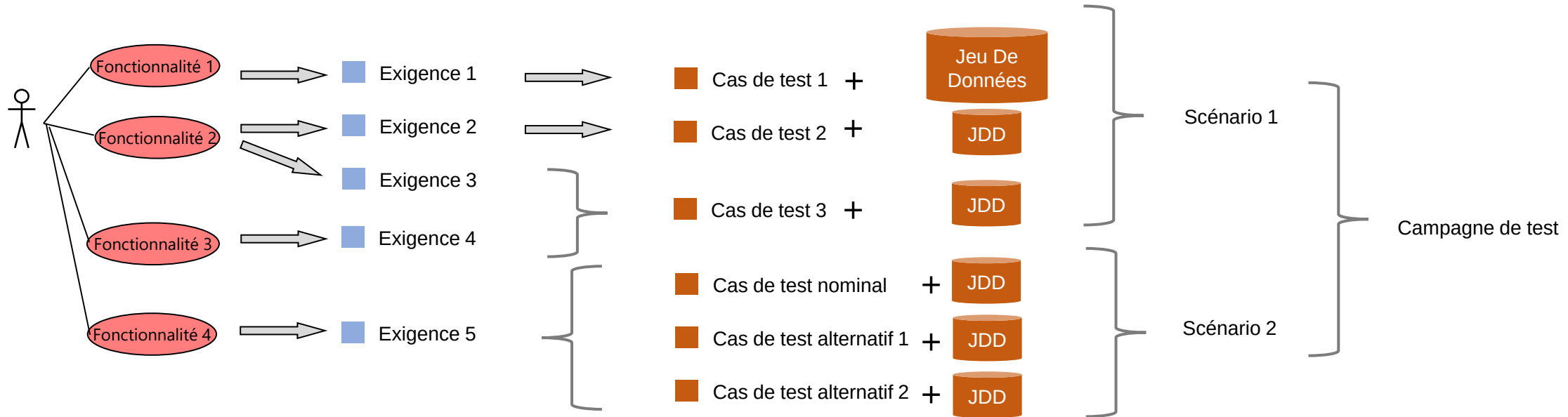
Selon le contexte, **l'approche équipe intégrée** n'est **pas toujours appropriée**. Par exemple, dans certaines situations, telles que les **situations critiques pour la sécurité**, un **niveau élevé d'indépendance du test** peut être **nécessaire**.

- Préparation de la recette fonctionnelle :

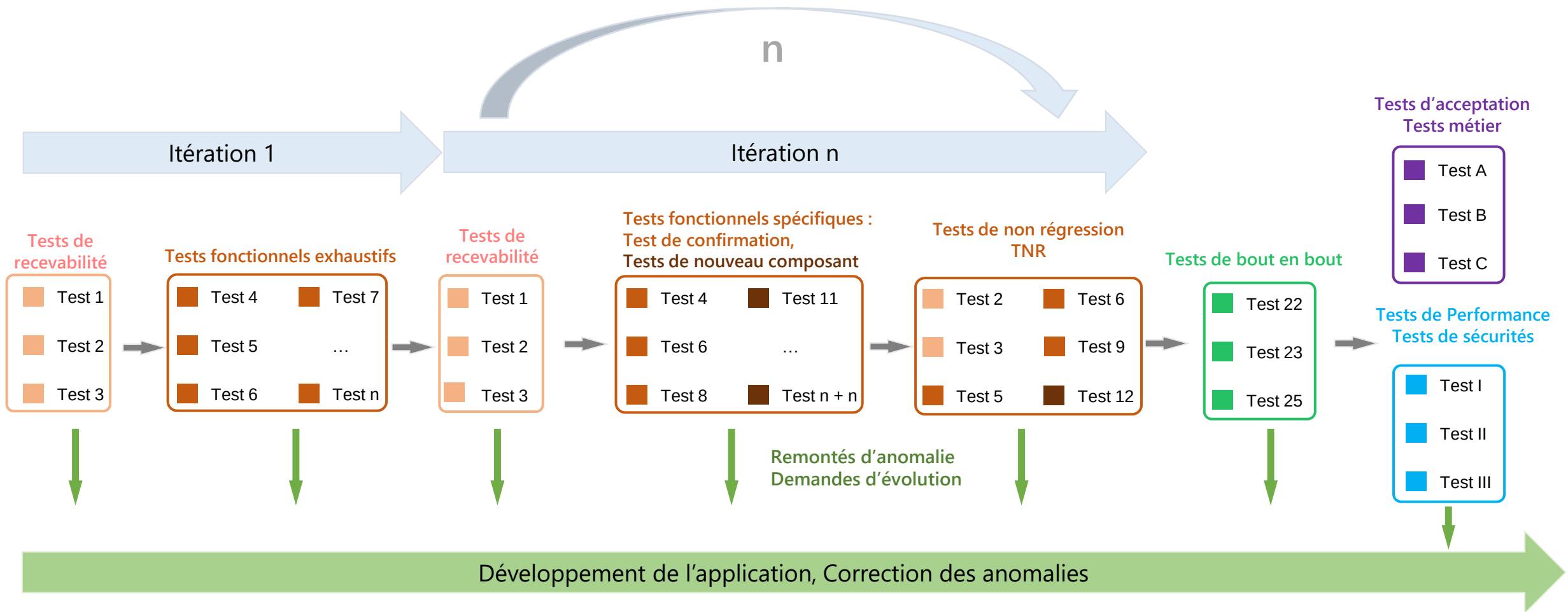
1- Formalisation des Exigences

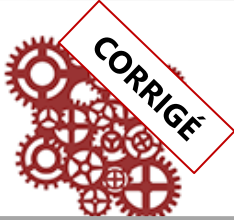
2- Création des Tests fonctionnels

3- Préparation du Plan d'Exécution



- Exécution de la recette fonctionnelle : Procéder de manière **itérative** et **incrémentale**



	Itération 1		CORRECTION		Itération 2	
Livraison initiale v.1.0	Exécution des tests système	Résultats	Relivraison v.1.1	Exécution des tests système	Tests de confirmation	Tests de régression
COMPOSANT 1		OK				
COMPOSANT 2		KO (Anomalie)				
COMPOSANT 3		OK				
			COMPOSANT 4			

- **Surface et profondeur de test :**

Dans le cadre du **test logiciel**, on distingue souvent :

- **Tester en surface** (ou tester en largeur)

On vérifie à un **niveau global** les principales fonctionnalités du produit.

Avantage : on détecte vite les **problèmes majeurs** (grosses régressions, blocages critiques) sur l'ensemble du logiciel.

Limite : on ne repère pas forcément les **bugs complexes** ou cachés, car les scénarios de test restent superficiels.

- **Tester en profondeur**

On se concentre sur un **périmètre restreint**, exploré en détail (cas d'erreur, conditions limites, combinaisons de données, stress, sécurité, etc.), pour identifier les failles potentielles ou les cas particuliers.

Avantage : on obtient une **forte confiance** sur la robustesse d'un composant sensible.

Limite : cela demande plus de **temps** et de **ressources**, et ne couvre pas l'ensemble du logiciel.

- **Comment ces approches se complètent :**

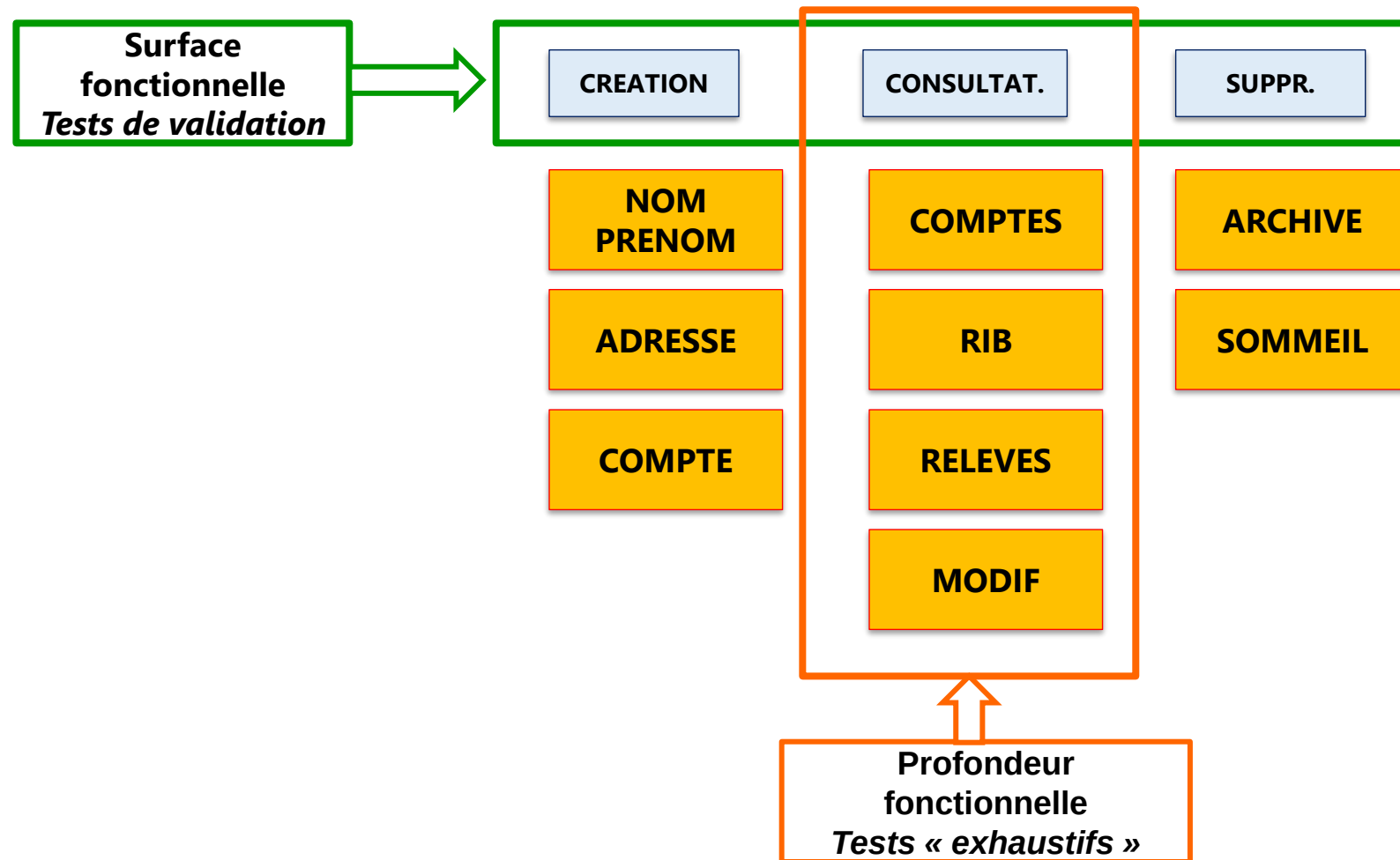
En pratique, on commence souvent par **tester en surface**, pour s'assurer que la solution fonctionne « à peu près » et détecter les éventuels blocages majeurs (smoke tests, vérifications de base).

Ensuite, on **cible** les zones les plus critiques ou à risque pour y appliquer des tests **en profondeur**.

Ainsi, on a à la fois **une vision large** (toute l'application est au moins testée sommairement) et **une vision précise** (les parties sensibles sont examinées de manière pointue).

Cette combinaison garantit un meilleur niveau de qualité qu'en s'appuyant uniquement sur une approche très large et superficielle, ou uniquement très pointue mais restreinte à quelques fonctionnalités.

- Surface et profondeur de test :



- **Conclusion**

- La qualité logicielle est une pratique :
 - indispensable
 - en constante évolution et amélioration
- Il n'existe pas de référence absolue, mais plutôt des **méthodes et bonnes pratiques reconnues** dans le monde de la qualité :
 - organismes : IEEE, NIST
 - normes / modèles : ISO 25.000, CMMI / TMMI, TMAP
 - certifications : ISTQB

Des auteurs ont écrit sur le sujet :

- Gerald Weinberg – *Série Quality software management*
- James Bach – *Lessons learned in software testing*
- Cem Kaner – *Testing computer software*